

Master Thesis

Recommendation of MOOCs using Graph Neural Networks

Zhang, Yan

3182349

First examiner: Prof. Dr. Mohamed Amine Chatti

Second examiner: Prof. Dr. Norbert Fuhr

Supervisor: M.Sc. Rawaa Alatrash

A thesis submitted to the Social Computing Group, Department of Human-centered Computing and Cognitive Science, Faculty of Computer Science at the University of Duisburg-Essen.

Duisburg, March 30, 2026

Erklärung

Hiermit versichere ich, dass ich diese Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

I hereby declare that I have created this work completely on my own and used no other sources or tools than the ones listed.

Duisburg, March 30, 2026

Yan Zhang

Acknowledgment

I would like to express my sincere gratitude to **Prof. Dr. Mohamed Amine Chatti** for his invaluable guidance, direction, and professional advice throughout my master's thesis. His support and feedback were of great help to me during the entire research and writing process. I would also like to sincerely thank **Rawaa Alatrash** for her kind, enthusiastic, and thoughtful support. She was always willing to help, and I truly appreciated her patience, encouragement, and continuous assistance. I am deeply grateful to both of them for the many meetings and discussions we had. Their support helped me greatly in developing, improving, and refining my thesis ideas.

Abstract

Personalized course recommendation is increasingly important in online learning platforms, where learners face a large number of available courses. A key challenge is how to construct a learner model that captures not only historical behavior but also the dynamic changes of learner interests over time. Existing approaches often rely mainly on user–item interactions and may not fully leverage learner-specific relationships and their temporal characteristics.

To address this problem, this thesis proposes LM-GNN, a learner model for personalized course recommendation based on learner-specific relationships. The proposed method integrates a heterogeneous knowledge graph, graph neural networks, and SBERT-based semantic representations[18] to construct learner embeddings. In particular, LM-GNN models three learner-specific relations, namely (Learner, *DNU*, Concept), (Learner, *INTERESTED_IN*, Concept), and (Learner, *ENGAGED_IN*, Course), in order to capture complementary aspects of learner behavior. To represent temporal dynamics, the model combines long-term interests and short-term interests within a unified framework. Long-term interests are modeled through a relation-specific sequential matrix that incorporates semantic correlation and temporal order, while short-term interests are modeled through position weights that emphasize recent interactions. Based on the resulting learner embedding, the system recommends courses from CourseMapper and MOOCCentral.

To examine the applicability of the proposed method, experiments and ablation studies are conducted on the MOOCCube dataset. Since MOOCCube provides only limited learner-specific relations and coarse-grained temporal signals, the evaluation mainly serves to analyze the conditions under which LM-GNN can be applied. The results do not show an advantage over existing supervised baselines on this dataset, but they suggest that the proposed learner modeling framework is closely related to the richness of learner-specific relationships and the granularity of behavioral and temporal information.

Overall, this thesis presents a learner-specific and multi-relationship learner modeling framework for course recommendation and provides insight into the data conditions under which such a model may be more suitable.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Thesis Aim and Research Questions	2
1.3	Thesis Outline	2
1.4	Thesis Outline	3
2	Related Work	5
2.1	CourseMapper Platform	5
2.2	PKG-Based Learner Modeling	5
2.3	GNN-Based Learner Modeling	6
2.4	ConceptGCN	7
2.5	RRGCN	7
2.6	Summary and Research Gap	8
3	Concept and Design	9
3.1	Platform and Database	9
3.1.1	Platform	9
3.1.1.1	CourseMapper Platform	10
3.1.1.2	MOOCCentral Platform	10
3.1.2	Database	11
3.1.2.1	CourseMapper Neo4j Database	11
3.1.2.2	MOOCCentral Neo4j Database	12
3.1.2.3	MongoDB Database	12
3.2	LM-GNN	13
3.2.1	Course Embedding Construction	13
3.2.1.1	Course Embedding Construction in CourseMapper	13
3.2.1.2	Course Embedding Construction in MoocCentral	14
3.2.2	Learner Model Construction Based on Historical Interactions with Learner-Specific Relationships	15
3.2.2.1	Construct Learner Model from Historical Interactions under Three Learner-Specific Relations	16
3.2.2.2	Calculate the Learner's Long-Term Interests under Three Learner-Specific Relations	18
3.2.2.3	Calculate the Learner's Short-Term interests under Three Learner-Specific Relations	22
3.2.2.4	Construct relation-specific weight matrices	23
3.2.2.5	Generate the Learner Model Embedding	24

3.2.3	Recommendation	25
4	Implementation	27
4.1	System Architecture & Technologies	27
4.2	Implementation of Course Embedding Construction	28
4.2.1	SBERT-based Name Embedding Generation	29
4.2.2	Construction of the Course–Concept Mapping	29
4.2.2.1	Course–Concept Mapping in MoocCentral	29
4.2.2.2	Course–Concept Mapping in CourseMapper	30
4.2.3	Course Embedding Construction	31
4.3	Implementation of Learner Model Embedding Construction	32
4.3.1	Construction of Relation Dictionaries	32
4.3.2	Computation of Temporal Indicators	35
4.3.3	Updating Node Embeddings Within Each Relation	36
4.3.4	Computation of Relation Components	38
4.3.5	Construction of the Final Learner Model Embedding	39
4.4	Implementation of the Variant-based Recommendation Pipeline	41
4.4.1	Offline Preprocessing and Embedding Preparation	42
4.4.2	Construction of the ENROLL Relation Dictionary	42
4.4.3	Construction of the INTEREST Relation Dictionary	45
4.4.4	Construction of Learner-Embedding Variants	46
4.4.5	Negative Sampling and Recommendation Generation	48
4.4.6	Evaluation Metrics	48
5	Evaluation	51
5.1	MOOCCube Dataset	51
5.2	Experimental Settings	51
5.3	Personal Knowledge Graph Construction in MOOCCube	52
5.4	Ablation Study	54
5.4.1	Generalized Learner Model Equation	54
5.4.2	Learner model Equation for MoocCube	55
5.4.2.1	Special Case: If User Has No Interested Concept	56
5.4.3	Variants based on the Generalized Learner model Equation	56
5.5	Evaluation Metrics	57
5.5.1	HR@K Equation 1 (Ground-Truth Level)	58
5.5.2	HR@K Equation 2 (User Level)	58
5.5.3	NDCG@K Equation	59
5.5.4	A Simple Example of Metric Calculation	59
5.5.4.1	HR@5 at the Ground-Truth Level	59
5.5.4.2	HR@5 at the User Level	60
5.5.4.3	NDCG@5	61
5.6	Baseline Methods	62
5.7	Discussion	63
5.7.1	Research Questions	63
5.7.2	Answer to RQ1	63
5.7.3	Answer to RQ2	64

5.7.4	Performance Analysis	65
5.7.4.1	Data sparsity in learner history	65
5.7.4.2	Limited learner-specific relationships in MOOCCube	66
5.7.4.3	Enrollment as a weak preference signal	66
5.7.4.4	Coarse-grained temporal signal weakens short-term preference modeling	67
5.7.4.5	Unsupervised learner modeling versus supervised ranking objectives	67
5.7.4.6	Ablation findings: simpler variants are more robust on MOOCCube	68
5.7.5	Summary of Discussion	68
6	Conclusion and Future Work	71
6.1	Conclusion	71
6.2	Future Work	72
	List of Tables	73
	List of Figures	75
	Bibliography	77
	Appendices	79
	Appendix 1	81

1 Introduction

Online learning platforms provide learners with access to a large and continuously growing number of courses, concepts, and learning materials. While this abundance of resources creates new opportunities for flexible learning, it also makes it more difficult for learners to identify courses that match their needs, interests, and knowledge state. As a result, personalized course recommendation has become an important task in online learning environments.

A central issue in personalized course recommendation is learner modeling. To recommend suitable courses, the system needs a representation of the learner that reflects what the learner has studied, what the learner is interested in, and what the learner may currently need. However, learner characteristics in online learning are not static. They evolve over time as learners interact with courses, concepts, and learning activities. Therefore, an effective learner model should not only summarize past behavior, but also capture the temporal dynamics of learner interests and learning status.

1.1 Motivation

Many existing recommendation methods are built mainly on user–course interaction data and focus on optimizing recommendation performance directly. Although such methods have achieved promising results, they often place less emphasis on modeling the learner through multiple learner-specific relationships. In real online learning platforms, learner behavior is more complex than simple enrollment or selection history. For example, a learner may show lack of understanding of certain concepts, may repeatedly interact with concepts of interest, and may demonstrate different levels of engagement in different courses. These behaviors reflect different aspects of the learner and may provide complementary signals for recommendation. Moreover, these signals are inherently temporal, since recent interactions may reflect current preferences, while repeated or semantically related historical interactions may reflect more stable long-term interests.

Motivated by this observation, this thesis proposes LM-GNN, a learner model for personalized course recommendation based on learner-specific relationships. The proposed method is designed for the CourseMapper platform and integrates a heterogeneous knowledge graph, graph neural networks, and SBERT-based semantic representations. In particular, LM-GNN models three learner-specific relations: (Learner, *DNU*, Concept), (Learner, *INTERESTED_IN*, Concept), and (Learner, *ENGAGED_IN*, Course). These relations are used to represent complementary aspects of learner behavior, including knowledge state, concept-level interest, and course-level engagement.

To capture temporal dynamics, LM-GNN jointly models long-term interests and short-term preferences under each learner-specific relations. Long-term interests are modeled through a relation-specific sequential matrix that combines semantic correlation among interacted nodes with the temporal order of interactions. Short-term preferences are modeled through position weights that assign greater importance to more recent interactions. Based on the resulting learner embedding, the system computes the similarity between learner embeddings and course embeddings in order to recommend courses from both CourseMapper and the MOOC-Central repository.

1.2 Thesis Aim and Research Questions

In addition to proposing the learner model, this thesis also examines its applicability under different data conditions. For this purpose, experiments and ablation studies are conducted on the MOOCCube dataset. Since MOOCCube does not provide the same richness of learner-specific relationships and fine-grained behavioral timestamps as CourseMapper, the evaluation is used not only to test the method, but also to analyze the extent to which the effectiveness of LM-GNN depends on the available relation types and temporal information.

Based on this objective, this thesis addresses the following research questions:

- **RQ1:** Can LM-GNN, which is constructed on learner-specific relationships, jointly model learners' long-term interests and short-term interests under different relations and produce a learner representation for course recommendation?
- **RQ2:** To what extent does the effectiveness of LM-GNN depend on the richness of learner-specific relationships and on the granularity of behavioral and temporal information? What are its applicability and limitations on a dataset such as MOOCCube, where available relations are limited and temporal signals are coarse-grained?

This thesis makes the following contributions. First, it proposes a learner-specific and multi-relationship learner modeling framework for personalized course recommendation. Second, it introduces a unified design that combines long-term interests and short-term preferences within the learner representation. Third, it analyzes the applicability of this learner model under restricted data conditions through experiments and ablation studies on MOOCCube.

1.3 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 3 presents the concept and design of LM-GNN, including the platform setting, database structure, learner model construction, and recommendation process. Chapter 5 describes the experimental setup, evaluation methodology, ablation study, and discussion. Finally, the last chapter concludes the thesis and outlines directions for future work.

1.4 Thesis Outline

The rest of this thesis will be divided into five different chapters, as follows:

- **Chapter 2, Background and Related Work.** This chapter introduces the Coursemapper platform, PKG-Based learner modeling, GNN-based learner modeling, ConceptGCN and RRGCN.
- **Chapter 3, Concept and Design.** presents the concept and design of LM-GNN, including the platform setting, database structure, course embedding construction, learner model construction, and recommendation process.
- **Chapter 4, Implementation.** This chapter describes the implementation of LM-GNN described in the previous chapter and key phases of evaluation chapter.
- **Chapter 5, Evaluation.** This chapter describes the experimental setup, evaluation metrics, baseline methods, ablation study, and discussion of the evaluation results.
- **Chapter 6, Conclusion and Future Work.** This chapter concludes the thesis and outlines directions for future work.

2 Related Work

This chapter reviews the work that is most relevant to this thesis. Since the proposed LM-GNN is developed in the context of online learning and graph-based learner modeling, the discussion focuses on the CourseMapper platform, personal knowledge graph (PKG)-based learner modeling, GNN-based learner modeling, ConceptGCN, and relation-aware graph neural networks such as RRGCN. Together, these studies provide the background and methodological foundation for the proposed approach.

2.1 CourseMapper Platform

CourseMapper is an online learning platform designed to support personalized and interactive learning [2]. It provides learners with access to courses, learning materials, concept-level knowledge graphs, and various collaborative learning functions. On the platform, learners can browse and study materials, explore related educational concepts, annotate learning content, take notes, mark concepts as understood or not understood, post comments, and reply to discussions. These interactions provide rich behavioral signals that can be used for learner modeling and personalized recommendation.

A distinctive feature of CourseMapper is its use of educational knowledge graphs to organize concepts and their relationships in a structured form. This graph-based organization makes it possible to connect learner behavior with semantic and structural information about the learning content. As a result, CourseMapper has become a suitable environment for research on graph-based learner modeling and recommendation. In particular, previous work on CourseMapper has shown that educational knowledge graphs and learner interaction data can be combined to support personalized concept recommendation [2]. In this thesis, CourseMapper serves as the main application context for developing a learner model for personalized course recommendation.

2.2 PKG-Based Learner Modeling

Knowledge Graphs (KGs) are widely used in user modeling because they can represent explicit relations between users, items, and other entities in a structured form. However, general KGs often contain large amounts of information that are not directly relevant to a specific user, which

limits their ability to support fine-grained personalization [21]. Personal Knowledge Graphs (PKGs) address this limitation by organizing only the entities and relations that are relevant to an individual user. In this way, PKGs provide a more focused representation of user knowledge, interests, and preferences, and therefore offer strong potential for personalized recommendation [3, 20].

In educational settings, PKGs have been explored for learner modeling and personalized support. Existing studies have shown that PKGs can be used to represent learner knowledge acquisition, learning progress, and concept-level interests [13]. In MOOC environments, prior work constructed a PKG for each learner by integrating learning resources, educational concepts, semantically related concepts, and learner–resource interaction data into a structured graph [1, 2]. This representation enables the system to capture learner interests at the concept level and supports personalized recommendation.

However, most existing PKG-based approaches in education rely on limited relation types, especially a single concept–concept relation such as *RELATED_TO* [2]. In practice, educational data often contain richer and more meaningful relations, both between educational entities and between learners and those entities. Therefore, extending PKGs with multiple relation types can preserve richer semantics and support more informative learner modeling. This idea is particularly important for downstream educational tasks such as personalized recommendation.

2.3 GNN-Based Learner Modeling

Graph Neural Networks (GNNs) have become an important method for representation learning on graph-structured data[8]. By propagating and aggregating information from neighboring nodes, GNNs can learn enhanced node representations that capture both local graph structure and higher-order neighborhood information. This makes them well suited to learner modeling on KGs and PKGs, where learners, concepts, resources, and courses are connected through meaningful relations.

In learner modeling, GNNs make it possible to move beyond isolated interaction records and represent learners in a broader graph-based context. Rather than relying only on direct learner–item interactions, GNN-based methods can incorporate information from related concepts, connected learning resources, and other neighboring entities. In this way, the learned representations can better reflect the learner’s interests, knowledge state, and behavioral patterns. Existing studies in recommendation have shown that graph-based methods can effectively utilize interaction structures and multi-hop relations to model user preferences.

In educational settings, GNN-based learner modeling has also attracted growing attention. Previous work on CourseMapper used Graph Convolutional Networks (GCNs) together with SBERT-based semantic representations to model learner knowledge state at the concept level and recommend relevant concepts [2, 18]. This line of work demonstrates that graph-based learner modeling can effectively combine educational knowledge structures with learner interaction data.

At the same time, many existing GNN-based learner modeling approaches do not explicitly distinguish different relation types during representation learning. When heterogeneous rela-

tions are treated in the same way, part of the relational semantics may be lost. This limitation is especially important in educational graphs, where different relations often reflect different pedagogical meanings and different aspects of learner behavior. Therefore, relation-aware GNNs are particularly relevant for learner modeling when the goal is to preserve richer semantics from multi-relational educational graphs.

2.4 ConceptGCN

ConceptGCN is a recommendation framework proposed for concept recommendation on the CourseMapper platform [2]. It combines an educational knowledge graph, Graph Convolutional Networks (GCNs), and SBERT-based semantic representations to generate personalized concept recommendations for learners [2, 18]. The main idea of ConceptGCN is to integrate structural information from the knowledge graph with semantic information from concept-related text, so that concept embeddings can better capture both graph context and semantic meaning [2].

On CourseMapper, ConceptGCN uses learner interactions together with concept-level graph structures to model learner interests and recommend relevant concepts. This provides an important foundation for personalized recommendation in online learning environments, since it shows how graph-based educational structures and learner behavior can be combined within a unified recommendation framework. In particular, ConceptGCN demonstrates the value of representing learners through their interactions with educational concepts and of enhancing concept representations through the integration of graph structure and semantic text information.

However, ConceptGCN mainly focuses on concept recommendation and does not explicitly model multiple learner-specific relations in an integrated way [2]. In addition, its learner modeling does not explicitly distinguish between long-term interests and short-term preferences over time. Therefore, while ConceptGCN provides an important basis for graph-based learner modeling on CourseMapper, further extensions are needed for richer learner modeling and course recommendation.

2.5 RRGCN

A central limitation of standard GCN-based methods is that they often aggregate neighboring information without explicitly distinguishing relation types. In heterogeneous graphs, however, different relations usually carry different semantics and should not be treated uniformly. This has motivated the development of relation-aware and multi-relational graph neural networks, which update node representations by taking relation type into account during message passing and aggregation [19, 23].

RRGCN is one example of a relation-aware graph neural network designed for multi-relational graphs [4]. Its core idea is that node embeddings should be updated not only according to neighboring nodes, but also according to the specific relations through which these neighbors

are connected. By incorporating relation-specific transformations, RRGCN can preserve richer relational semantics during representation learning and is therefore more suitable than relation-agnostic GCNs for heterogeneous graph settings [4].

This idea is particularly relevant in educational graphs, where relations may reflect different semantic or pedagogical meanings. In the context of this thesis, RRGCN is important because the concept representations used in the learner model are constructed from a multi-relational graph setting. More specifically, the embedding of a concept is obtained by aggregating information from neighboring nodes connected through different relation types. In addition, the idea of relation-specific transformation in RRGCN also motivates the use of relation-specific weight matrices in the learner modeling framework proposed in this thesis. Therefore, RRGCN provides an important methodological basis for relation-aware representation learning in this work.

2.6 Summary and Research Gap

The reviewed studies provide several important foundations for this thesis. CourseMapper offers an online learning environment in which learner behavior and educational concepts can be represented in graph form. PKG-based learner modeling shows how user-specific graph structures can support personalization. GNN-based learner modeling further demonstrates how graph structure can be used to enhance learner and concept representations. ConceptGCN illustrates the value of combining educational knowledge graphs, GCNs, and semantic encoders for recommendation on CourseMapper. Relation-aware models such as RRGCN highlight the importance of preserving relation-specific semantics in heterogeneous graphs.

At the same time, a gap remains in the literature. Existing work does not fully address learner modeling for course recommendation based on multiple learner-specific relations and temporal interaction dynamics. In particular, prior approaches either focus mainly on concept recommendation, rely on limited relation types, or do not explicitly distinguish long-term and short-term learner preferences. Motivated by this gap, the proposed LM-GNN extends prior work by integrating multiple learner-specific relations, relation-aware graph-based representations, and temporal preference modeling into a unified learner modeling framework for personalized course recommendation.

3 Concept and Design

This chapter introduces the proposed learner model approach, namely Learner Model based on Graph Neural Network (LM-GNN), for personalized course recommendation in online learning platforms. The method integrates a heterogeneous knowledge graph, a graph neural network (GNN), and a pre-trained language model encoder (SBERT) [18] to construct learner embedding by combining historical behavioral data with multiple relation types associated with learners.

The overall architecture of the model is illustrated in Fig.3.1. The method operates in an on-line phase, where learner model embedding is updated based on real-time behaviors to continuously improve recommendation results. The learner model is implemented on the CourseMapper platform and makes use of structural information from a personal knowledge graph. The design is inspired by ConceptGCN [2] and a Multi-relational-GCN-RRGCN (MR-GCN-RRGCN) proposed by a master student[24], which is able to model different types of relation. The model captures learner behavior by integrating historical interactions with courses and concepts, together with multiple learner-related relations.

Specifically, the model considers three types of relations, namely learner engagement in courses, learner interest in concepts, and learner marking of concepts as DID NOT UNDERSTAND (DNU). These relations correspond to knowledge state, engagement level, and interest level, and together form a comprehensive learner representation. Based on this representation, the system recommends courses by matching learner embeddings with candidate courses from CourseMapper and MOOCCentral.

3.1 Platform and Database

3.1.1 Platform

This section introduces the CourseMapper platform and the 10 MOOC platforms included in the MOOCCentral database.

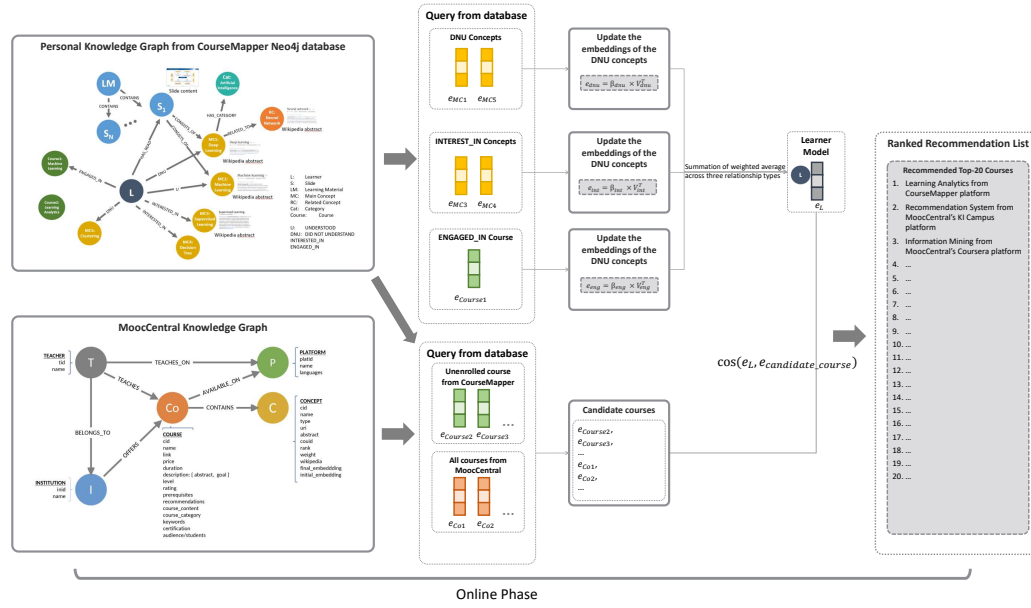


Figure 3.1: The Conceptual Architecture of LM-GNN

3.1.1.1 CourseMapper Platform

The proposed LM-GNN method is implemented on the CourseMapper online learning platform, which provides rich interaction data for learner modeling. On this platform, learners can enroll in courses and participate in various learning activities. These activities include browsing and watching learning materials, exploring concept-level knowledge graphs related to the materials, marking concepts as understood or not understood, posting comments, liking, and replying within enrolled courses, and annotating or taking notes on learning content. These behaviors reflect learner states from multiple perspectives. In particular, DNU marks indicate the level of understanding of knowledge, while interaction behaviors such as commenting and replying reflect the engagement of the learner. At the same time, these actions also provide signals of learner interest in specific concepts. Such multi-dimensional behavioral data form an important foundation for constructing learner representations.

3.1.1.2 MOOCentral Platform

To extend the coverage of available courses, this study also incorporates data from the MOOCentral database. MOOCentral includes 10 MOOC platforms, namely KI Campus, onCampus, OpenHPI, OpenVHB, edX, FutureLearn, Coursera, iMooX, Udemy, and Udacity. These platforms provide 33,638 courses, which together form a large candidate set for recommendation and help improve recommendation diversity.

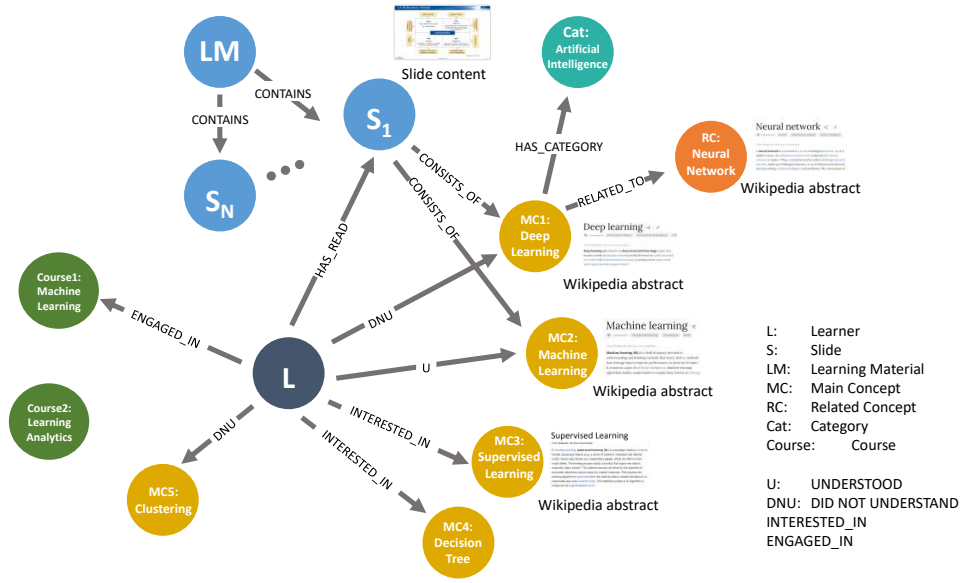


Figure 3.2: Personal knowledge graph in the CourseMapper database.

3.1.2 Database

In this section, we introduce the three databases that support learner modeling and course recommendation.

3.1.2.1 CourseMapper Neo4j Database

The system uses three databases, namely a Neo4j graph database for CourseMapper, a Neo4j graph database for MOOCentral, and a MongoDB database, to support different data storage and processing needs.

The CourseMapper Neo4j graph database stores the personal knowledge graph, as shown in Fig. 3.2. It includes following entities: Learner (L), Learning Material (LM), Slide (S), Main Concepts (MC), Related Concept (RC), Category (Cat), Course (Course). These nodes are connected through relations that describe learner interactions and semantic links. The learner model captures three learner-specific relationships, namely (L, *DNU*, Concept), (L, *INTERESTED_IN*, Concept) and (L, *ENGAGED_IN*, Course). These relationships constitute the core foundation of learner modeling, as they capture complementary aspects of learner behavior and learning status.

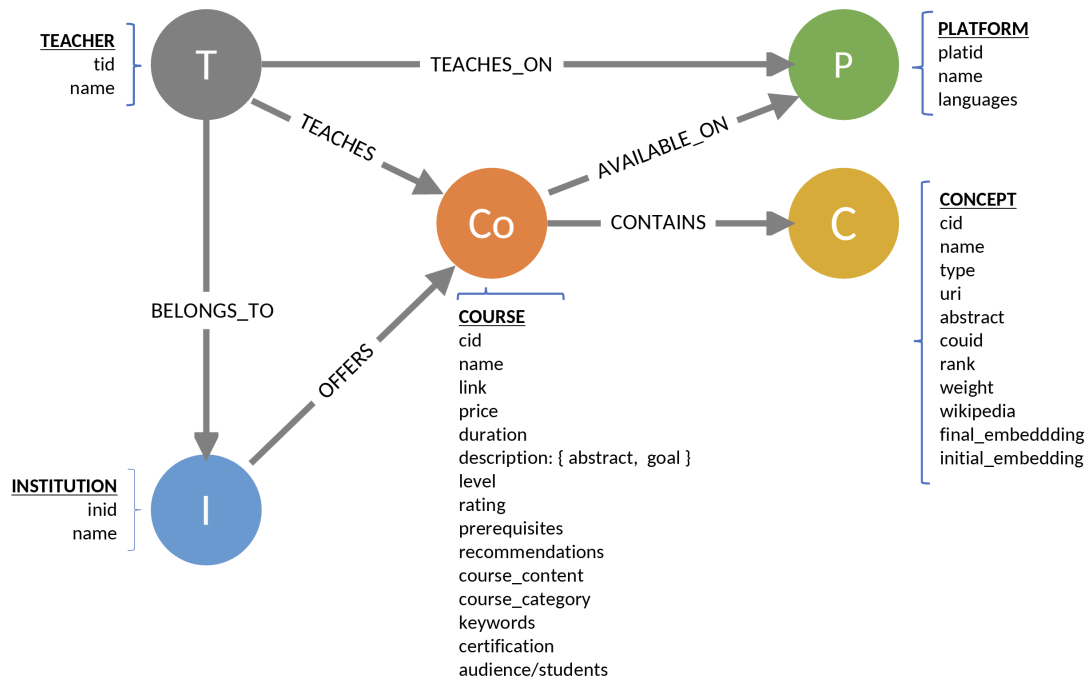


Figure 3.3: Knowledge graph schema of the MOOCCentral database.

3.1.2.2 MOOCCentral Neo4j Database

The MOOCCentral Neo4j graph database provides large-scale course resources and is organized as a knowledge graph, as shown in Fig. 3.3. It includes 10 Platform nodes, 33,638 Course nodes, and 858 Concept nodes. These entities are connected through relations such as (INSTITUTION, *OFFERS*, COURSE), (TEACHER, *BELONGS_TO*, INSTITUTION), (TEACHER, *TEACHES*, COURSE), (TEACHER, *TEACHES_ON*, PLATFORM), (COURSE, *AVAILABLE_ON*, PLATFORM) and (COURSE, *CONTAINS*, CONCEPT). Each platform provides a set of course resources, and together they form a large candidate pool for recommendation, which improves both coverage and diversity.

3.1.2.3 MongoDB Database

The MongoDB database stores online behavioral data of learners and supports efficient storage of real-time interaction logs. In particular, the system records DNU concept interactions with timestamps for each learner. These data support the dynamic construction of learner representations during the online phase.

3.2 LM-GNN

This section introduces the workflow of LM-GNN. We first construct course embeddings for all courses in the CourseMapper and MOOCCentral graph databases. We then construct learner model embeddings by integrating three types of relationships, namely (L, *DNU*, Concept), (L, *INTERESTED_IN*, Concept), and (L, *ENGAGED_IN*, Course), with the historical interactions of users on the CourseMapper platform while preserving the temporal order of these interactions. This design enables the incorporation of temporal information from learner interactions and supports the modeling of both long-term and short-term learner interests. Finally, for each user, we generate a personalized course recommendation list by ranking the predicted matching scores between course embeddings and learner model embeddings.

3.2.1 Course Embedding Construction

This subsection describes the construction of course embeddings for courses on the CourseMapper platform and for courses in the MOOCCentral database. Inspired by prior studies [25, 17], our method emphasizes the relationships between a course and the concepts covered by that course in order to construct course representations. We first describe how course embeddings are constructed for CourseMapper courses and then present the corresponding construction process for MOOCCentral courses. The resulting course embeddings are used in the recommendation stage as the embeddings of candidate courses for generating personalized recommendation lists for learners.

3.2.1.1 Course Embedding Construction in CourseMapper

For a given course *course* on the CourseMapper platform, the construction of the course embedding consists of three steps. (1) We use SBERT to encode the course name and obtain a name-based course embedding for course *course*. (2) We retrieve from the CourseMapper database all concepts covered by course *course*, and for each retrieved concept, we use the embedding produced by MR-GCN-RRGCN. Specifically, this concept embedding is obtained by aggregating information from neighboring nodes connected to the target concept through multiple relationship types, thereby integrating information from the two-hop neighborhood of that concept in the knowledge graph. (3) Based on the embeddings of all concepts associated with course *course*, we compute the weight of each concept embedding by measuring the cosine similarity between the course name embedding and the corresponding concept embedding. We then normalize these weights by their sum and compute the weighted combination of all concept embeddings to obtain the embedding of course *course*.

The course embedding equation in CourseMapper is defined as follows:

$$e_{course} = \frac{1}{w_{sum}} \sum_{c_i \in C(course)} w_i e_{c_i}, \quad (3.1)$$

where

$$\begin{aligned} e_{course_name} &= \text{SBERT}(course_name), \\ w_i &= \cos(e_{course_name}, e_{c_i}), \\ w_{sum} &= \sum_{c_i \in C(course)} w_i. \end{aligned}$$

The components in the above equation are defined as follows:

- e_{course} denotes the course embedding.
- e_{course_name} denotes the course name embedding obtained from the course name using SBERT.
- $C(course)$ denotes the set of concepts contained in course $course$.
- e_{c_i} denotes the embedding of concept c_i obtained using MR-GCN-RRGCN.
- w_i denotes the cosine similarity weight assigned to concept c_i .
- w_{sum} denotes the sum of the weights of all concepts associated with course $course$ and serves as the normalization factor.

3.2.1.2 Course Embedding Construction in MoocCentral

For a given course $course$ in the MoocCentral database, the construction of the course embedding also consists of three steps. (1) We use SBERT to encode the course name and obtain a name-based course embedding for course $course$. (2) We retrieve from the MOOC-Central database all concepts covered by course $course$, and for each retrieved concept, we use SBERT to encode the concept name and obtain the corresponding concept embedding. (3) Based on the embeddings of all concepts associated with course $course$, we compute the weight of each concept embedding by measuring the cosine similarity between the course name embedding and the corresponding concept embedding. We then normalize these weights by their sum and compute the weighted combination of all concept embeddings to obtain the embedding of course $course$.

The course embedding equation in MoocCentral is defined as follows:

$$e_{course} = \frac{1}{w_{sum}} \sum_{c_i \in C(course)} w_i e_{c_i}, \quad (3.2)$$

where

$$\begin{aligned}
 e_{course_name} &= \text{SBERT}(course_name), \\
 e_{c_i} &= \text{SBERT}(concept_name_i), \\
 w_i &= \cos(e_{course_name}, e_{c_i}), \\
 w_{sum} &= \sum_{c_i \in C(course)} w_i.
 \end{aligned}$$

The components in the above equation are defined as follows:

- e_{course} denotes the course embedding.
- e_{course_name} denotes the course name embedding obtained from the course name using SBERT.
- $C(course)$ denotes the set of concepts contained in course $course$.
- e_{c_i} denotes the embedding of concept c_i obtained from the concept name using SBERT.
- w_i denotes the cosine similarity weight assigned to concept c_i .
- w_{sum} denotes the sum of the weights of all concepts associated with course $course$ and serves as the normalization factor.

3.2.2 Learner Model Construction Based on Historical Interactions with Learner-Specific Relationships

In online learning environments, the knowledge state, concept interest preference, and course engagement of a learner are not fixed. Instead, they change continuously as the learner interacts with the learning system over time. To capture these dynamic characteristics in a more fine-grained way, we construct a learner model in the online phase based on multi-relational sequential interactions. Different from ConceptGCN[2] that rely on only the (L, *DNU*, Concept), our method considers three learner-specific relations, namely (L, *DNU*, Concept), (L, *INTERESTED_IN*, Concept) and (L, *ENGAGED_IN*, Course). These three relations describe the learner knowledge state, concept-level interests, and course-level engagement, respectively. Modeling the learner through multiple learner-specific relation types may offer a more complete description of the learner than single-relation modeling. To capture the temporal dynamics of interactions under different learner-specific relations, we further incorporate temporal information into the learner modeling process. This design allows the model to capture both long-term and short-term interests of the learner. Specifically, long-term interests are modeled through the semantic correlations and sequential order of historical interactions within each relation, while short-term interests are modeled through position weights that emphasize the influence of recent interactions on the current learner representation. To achieve this, our learner modeling approach involves four steps: (1) constructing the learner model based on the learner's historical interactions under different learner-specific relation types; (2) modeling the learner's long-term interests and updating the embeddings of the interacted nodes by incorporating semantic and sequential information within each relation type; (3) modeling the learner's short-term interests for each learner-specific relation type; and (4) introducing a randomly ini-

tialized relation-specific weight matrix for each relation type, and (5) generating the final learner model embedding.

3.2.2.1 Construct Learner Model from Historical Interactions under Three Learner-Specific Relations

We construct the learner model based on the continuous interaction records collected from the online platform across three learner-specific relations during learning. On CourseMapper, a learner can mark concepts in learning materials as DNU or U, annotate learning materials, ask questions in courses, and reply to discussions. When a learner marks a concept as DNU in the online phase, the system directly records this DNU event in the database together with its timestamp. For the interest relation and the engagement relation, the system automatically invokes the interest module and the engagement module every day at 11:00 p.m. to recalculate the learner's interest scores for concepts and engagement levels for courses. If the interest score of a concept changes, the timestamp of that concept under the interest relation is updated in the database. If the engagement level of a course changes, the timestamp of that course under the engagement relation is updated accordingly. Since DNU, interest, and engagement capture different aspects of learner behavior, their records are maintained separately. Therefore, a timestamp is always attached to a specific relation rather than to the entity itself. Even if the same concept appears in both the DNU relation and the interest relation, the timestamps in these two relations describe different events and must be stored and modeled independently.

To illustrate how records are maintained under different learner-specific relations, we use an example centered on a learner and the multiple relations associated with that learner, as shown in Fig. 3.4. We assume that the timestamps increase with the order of events. Figure 3.4(a) shows the PKG before the first automatic recalculation. At this point, the database already stores several historical records under the engagement relation and the interest relation, while no record has yet been created under the DNU relation. Fig. 3.4(b) shows the PKG after the first automatic recalculation. At time t_5 , the system finds that only the interest score of Concept 2 has changed. As a result, the record of Concept 2 under the interest relation is updated with the new interest score and the new timestamp t_5 , while all other relation records remain unchanged.

After that, the learner performs two explicit DNU actions. Specifically, the learner marks Concept 1 as DNU at time t_6 and Concept 3 as DNU at time t_7 . These two interactions create two new records under the DNU relation, as shown in Fig. 3.4(c). Later, at time t_8 , the system performs the second automatic recalculation. During this recalculation, the system detects that the engagement level of Course 2 has changed, and it also detects changes in the interest scores of Concept 1 and Concept 3. Therefore, the corresponding records under the engagement relation and the interest relation are updated with new timestamps. The resulting graph is shown in Fig. 3.4(d), which represents the current personalized knowledge graph of the learner.

Based on the final PKG shown in Fig. 3.4(d), Table 3.1 summarizes the current PKG snapshot of the learner. For each learner, the personalized knowledge graph consists of multiple learner-specific relations, and each relation maintains its own records and timestamps. As a result, the same entity may be associated with different timestamps under different relations. For example,

Table 3.1: Learner-specific relation timestamp records for online modeling

Relation type	Entity	State value	Timestamp
DNU	Concept 1	1	t_6
DNU	Concept 3	1	t_7
INTERESTED_IN	Concept 1	0.7	t_8
INTERESTED_IN	Concept 2	0.5	t_5
INTERESTED_IN	Concept 3	0.6	t_8
ENGAGED_IN	Course 1	low	t_1
ENGAGED_IN	Course 2	medium	t_8

Note: The *state value* has different meanings for different relation types. For DNU, a value of 1 indicates that the corresponding concept is marked as DNU. For INTERESTED_IN, the state value denotes the learner's interest score for the concept. For ENGAGED_IN, categorical values such as *low*, *medium*, and *high* indicate the learner's engagement level with the course.

the timestamp of Concept 1 under the DNU relation is t_6 , while its timestamp under the interest relation is t_8 . The former indicates when the concept is marked as not understood, whereas the latter indicates when the interest score of that concept is updated most recently. Therefore, the following modeling process is performed independently within each learner-specific relation, rather than by mixing all interaction events into a single unified sequence.

If no new interaction occurs after the current moment, the timestamp records in Table 3.1 can be regarded as a complete description of learner L under multiple learner-specific relationships. Based on these records, we represent learner L by three learner-specific relationship sequences, namely $(L, \text{DNU}, \text{Concept})$, $(L, \text{INTERESTED_IN}, \text{Concept})$ and $(L, \text{ENGAGED_IN}, \text{Course})$ relationships, respectively. These three sequences can be summarized as

$$L = \{L_{dnu}, L_{int}, L_{eng}\}. \quad (3.3)$$

For the example in Table 3.1, the three learner-specific relationship sequences of learner L can be written as

$$L_{dnu} = [(1, t_6)_{\text{Concept}_1}, (1, t_7)_{\text{Concept}_3}], \quad (3.4)$$

$$L_{int} = [(1, t_8)_{\text{Concept}_1}, (1, t_5)_{\text{Concept}_2}, (1, t_8)_{\text{Concept}_3}], \quad (3.5)$$

$$L_{eng} = [(1, t_1)_{\text{Course}_1}, (1, t_8)_{\text{Course}_2}]. \quad (3.6)$$

In all three learner-specific relationship sequences, the first value 1 indicates that the corresponding node is currently associated with learner L under that relationship. More specifically, in L_{DNU} , t_6 and t_7 denote the timestamps of Concept1 and Concept3 under the DNU relationship, indicating when these two concepts are marked as DNU by learner L . In L_{INT} , t_8 , t_5 , and t_8 denote the timestamps of Concept1, Concept2, and Concept3 under the interest relationship, indicating the most recent updates of their interest scores. In L_{ENG} , t_1 and t_8 denote the timestamps of Course1 and Course2 under the engagement relationship, indicating the most recent updates of their engagement levels. By preserving these relation-dependent timestamps, the learner model can better reflect how the learner's knowledge state, interests, and course engagement evolve over time.

3.2.2.2 Calculate the Learner's Long-Term Interests under Three Learner-Specific Relations

To capture the learner's long-term interests, we model the sequential correlations among interacted nodes under each learner-specific relation. Inspired by RetaGNN [12], we assume that long-term interests can be reflected not only by the semantic relatedness among interacted nodes, but also by their temporal order in the learner's historical interaction sequence. Our method considers three learner-specific relation sequences, namely (L, *DNU*, Concept), (L, *INTERESTED_IN*, Concept) and (L, *ENGAGED_IN*, Course). For each relation sequence, we construct a relation-specific sequential matrix to capture the semantic correlation and chronological order among the interacted nodes, and then use it to update the corresponding node embeddings.

In the online phase, long-term interest modeling is performed without relying on labeled data or additional training. Instead, we focus on identifying which interacted nodes are semantically close and how they are ordered in each learner-specific historical sequence. In this way, the model can update the node representations under each relation according to both semantic information and sequential information. As a result, nodes that are more semantically relevant and more consistent with the learner's historical interaction pattern can contribute more to the final learner representation.

This intuition can be understood through a simple example. Suppose that, under the *DNU* relation, the learner's historical *DNU* concepts in chronological order are supervised learning, IoT, and Bayesian classifier. By calculating cosine similarity, we find that supervised learning and Bayesian classifier have a relatively high similarity, whereas IoT has a relatively low similarity to both of them. This suggests that the learner's historical *DNU* concepts are more concentrated in the machine learning topic. Therefore, the semantically closer *DNU* concepts can be assigned higher weight so that these concepts contribute more to the final learner representation. The same intuition also applies to the other learner-specific relations. Under the interest relation, semantically similar concepts may indicate a stable topic preference of the learner. Under the engagement relation, semantically similar courses may reflect a sustained learning focus at the course level. In this sense, modeling semantic correlation within each relation helps the learner model capture long-term interests and persistent behavior patterns more effectively.

At the same time, long-term interests modeling also needs to consider the temporal order of interactions, since learner behavior is inherently sequential. For each learner-specific relation, we assume that the historical interactions follow a causal order, where past interactions can influence future ones, but future interactions cannot influence past ones. This assumption is consistent with the natural order of learner behavior over time. Inspired by RetaGNN [12], we introduce a mask matrix, denoted by I , to preserve this sequential constraint in each learner-specific relation sequence. More specifically, the mask matrix ensures that past interactions can affect future interactions, while future interactions cannot affect past interactions. In this way, the model can better capture the temporal order of the learner's historical interactions. Combined with the correlation matrix, this temporal constraint forms a relation-specific sequential matrix, which is then used to update the embeddings of the interacted nodes under the corresponding learner-specific relation.

Based on the above design, long-term interests modeling under each learner-specific relation

relies on three components, namely the relation-specific embedding matrix obtained from the offline phase, the correlation matrix that captures semantic similarity, and the mask matrix that preserves temporal order. Specifically, for the DNU and interest relations, the embeddings of the interacted concepts are derived from the MR-GCN-RRGCN concept representations stored in the CourseMapper Neo4j database, as introduced in Section 3.2.1.1. For the engagement relation, the embeddings of the interacted courses are derived from the course embeddings defined in Equation 3.1. In the following, we formally define these components and show how they are combined into the sequential matrix for updating node embeddings under the three learner-specific relations.

Formally, for each learner-specific relation $r \in \{dnu, int, eng\}$, we update the embeddings of the interacted nodes by using a relation-specific sequential matrix β_r as follows:

$$e_r = \beta_r \cdot V_r^\top \quad (3.7)$$

where,

- e_r denotes the output matrix under learner-specific relation r , which is composed of the updated embeddings of the interacted nodes under relation r , i.e.,

$$e_r = [e_{C_1}^r, e_{C_2}^r, \dots, e_{C_{|N_L^r|}}^r] \quad (3.8)$$

where $C_i \in N_L^r$ denotes a node associated with learner L under relation r . Here, c_i is a concept node when $r \in \{dnu, int\}$, and a course node when $r = eng$.

- $V_r^\top$ denotes the embedding matrix of the time-ordered interacted nodes under learner-specific relation r , obtained from the offline phase, i.e.,

$$V_r^\top = [e_{t_{C_1}}^{C_1}, e_{t_{C_2}}^{C_2}, \dots, e_{t_{C_{|N_L^r|}}}^{C_{|N_L^r|}}] \quad (3.9)$$

where $e_{t_{C_i}}^{C_i}$ denotes the embedding of node C_i before the online update under relation r and t_{C_i} is corresponding timestamp of node C_i under relation r

- β_r denotes the sequential matrix under learner-specific relation r . It is calculated as

$$\beta_r = C_r + I_r \quad (3.10)$$

and each entry $-\infty$ in β_r is replaced with 0 after the addition. C_r is the correlation matrix and I_r is the mask matrix under relation r .

Here, C_r denotes the correlation matrix under learner-specific relation r , which is computed by cosine similarity between all pairs of interacted nodes:

$$(C_r)_{ij} = \cos(e_{t_i}^{C_x}, e_{t_j}^{C_y}) \quad (3.11)$$

where $e_{t_i}^{C_x}$ and $e_{t_j}^{C_y}$ denote the embeddings of nodes C_x and C_y under learner-specific relation r , respectively, and t_i and t_j denote their corresponding timestamps under that relation.

The matrix I_r denotes the mask matrix under learner-specific relation r , which preserves

the chronological order of interactions:

$$(I_r)_{ij} = \begin{cases} 0, & t_i \leq t_j \\ -\infty, & t_i > t_j \end{cases} \quad (3.12)$$

where t_i and t_j denote the timestamps of nodes C_x and C_y under learner-specific relation r , respectively.

Referring to the node embeddings and their timestamps in a learner-specific relation sequence, the mask matrix can be interpreted as preserving the causal order of interactions under that relation. If a node is interacted with earlier than another node, the earlier interaction is allowed to influence the later one. In contrast, if a node appears later in the sequence, it is not allowed to affect nodes that appear earlier. In this way, the mask matrix ensures that the sequential matrix respects the chronological structure of the learner's historical interactions.

For the DNU relation in Table 3.1, the learner-specific relation sequence L_{dnu} is defined in Eq. 3.4. Since $t_6 < t_7$, the interaction on Concept₁ under the DNU relation occurs earlier than that on Concept₃. Therefore, according to Eq. 3.12, the earlier DNU interaction on Concept₁ can influence the later DNU interaction on Concept₃, whereas the reverse direction is not allowed. Based on Eq. 3.9, the embedding matrix of the time-ordered interacted nodes under the DNU relation is

$$V_{dnu}^\top = [e_{t_6}^{Concept_1}, e_{t_7}^{Concept_3}]. \quad (3.13)$$

According to Eq. 3.11, the corresponding correlation matrix is

$$C_{dnu} = \begin{bmatrix} 1 & \cos(e_{t_6}^{Concept_1}, e_{t_7}^{Concept_3}) \\ \cos(e_{t_7}^{Concept_3}, e_{t_6}^{Concept_1}) & 1 \end{bmatrix}. \quad (3.14)$$

According to Eq. 3.12, the corresponding mask matrix is

$$I_{dnu} = \begin{bmatrix} 0 & 0 \\ -\infty & 0 \end{bmatrix}. \quad (3.15)$$

Therefore, the sequential matrix under the DNU relation is

$$\beta_{dnu} = C_{dnu} + I_{dnu}. \quad (3.16)$$

After replacing each $-\infty$ entry with 0, we obtain

$$\beta_{dnu} = \begin{bmatrix} 1 & \cos(e_{t_6}^{Concept_1}, e_{t_7}^{Concept_3}) \\ 0 & 1 \end{bmatrix}. \quad (3.17)$$

Finally, the updated embeddings under the DNU relation are computed according to Eq. 3.7 as

$$e_{dnu} = \beta_{dnu} \cdot V_{dnu}^\top. \quad (3.18)$$

For the interest relation in Table 3.1, the learner-specific relation sequence L_{int} is defined in Eq. 3.5. Following the timestamps in chronological order, the corresponding node sequence is Concept₂ at t_5 , Concept₁ at t_8 , and Concept₃ at t_8 . Accordingly, the embedding matrix under

the interest relation is

$$V_{int}^\top = [e_{t_5}^{Concept_2}, e_{t_8}^{Concept_1}, e_{t_8}^{Concept_3}]. \quad (3.19)$$

The corresponding correlation matrix is

$$C_{int} = \begin{bmatrix} 1 & \cos(e_{t_5}^{Concept_2}, e_{t_8}^{Concept_1}) & \cos(e_{t_5}^{Concept_2}, e_{t_8}^{Concept_3}) \\ \cos(e_{t_8}^{Concept_1}, e_{t_5}^{Concept_2}) & 1 & \cos(e_{t_8}^{Concept_1}, e_{t_8}^{Concept_3}) \\ \cos(e_{t_8}^{Concept_3}, e_{t_5}^{Concept_2}) & \cos(e_{t_8}^{Concept_3}, e_{t_8}^{Concept_1}) & 1 \end{bmatrix}. \quad (3.20)$$

Since $t_5 < t_8 = t_8$, the mask matrix under the interest relation is

$$I_{int} = \begin{bmatrix} 0 & 0 & 0 \\ -\infty & 0 & 0 \\ -\infty & 0 & 0 \end{bmatrix}. \quad (3.21)$$

Thus, the sequential matrix under the interest relation is

$$\beta_{int} = C_{int} + I_{int}, \quad (3.22)$$

and after replacing each $-\infty$ entry with 0, we obtain

$$\beta_{int} = \begin{bmatrix} 1 & \cos(e_{t_5}^{Concept_2}, e_{t_8}^{Concept_1}) & \cos(e_{t_5}^{Concept_2}, e_{t_8}^{Concept_3}) \\ 0 & 1 & \cos(e_{t_8}^{Concept_1}, e_{t_8}^{Concept_3}) \\ 0 & \cos(e_{t_8}^{Concept_3}, e_{t_8}^{Concept_1}) & 1 \end{bmatrix}. \quad (3.23)$$

The updated embeddings under the interest relation are then computed as

$$e_{int} = \beta_{int} \cdot V_{int}^\top. \quad (3.24)$$

For the engagement relation in Table 3.1, the learner-specific relation sequence L_{eng} is defined in Eq. 3.6. Since $t_1 < t_8$, $Course_1$ appears earlier than $Course_2$ under the engagement relation. Accordingly, the embedding matrix under this relation is

$$V_{eng}^\top = [e_{t_1}^{Course_1}, e_{t_8}^{Course_2}]. \quad (3.25)$$

The correlation matrix is

$$C_{eng} = \begin{bmatrix} 1 & \cos(e_{t_1}^{Course_1}, e_{t_8}^{Course_2}) \\ \cos(e_{t_8}^{Course_2}, e_{t_1}^{Course_1}) & 1 \end{bmatrix}. \quad (3.26)$$

The mask matrix is

$$I_{eng} = \begin{bmatrix} 0 & 0 \\ -\infty & 0 \end{bmatrix}. \quad (3.27)$$

Thus, the sequential matrix under the engagement relation is

$$\beta_{eng} = C_{eng} + I_{eng}, \quad (3.28)$$

and after replacing each $-\infty$ entry with 0, we obtain

$$\beta_{eng} = \begin{bmatrix} 1 & \cos(e_{t_1}^{Course_1}, e_{t_8}^{Course_2}) \\ 0 & 1 \end{bmatrix}. \quad (3.29)$$

Finally, the updated embeddings under the engagement relation are computed as

$$e_{eng} = \beta_{eng} \cdot V_{eng}^\top. \quad (3.30)$$

After the above operations, we obtain the updated node embeddings under the (L, *DNU*, Concept), (L, *INTERESTED_IN*, Concept) and (L, *ENGAGED_IN*, Course) relations, respectively. In this way, the learner's long-term interests under different learner-specific relations are captured through both semantic correlation and temporal order. These updated relation-specific embeddings are then used in the next step to model the learner's short-term interests and to generate the final learner representation.

3.2.2.3 Calculate the Learner's Short-Term interests under Three Learner-Specific Relations

To further capture the learner's short-term interests, we introduce relation-specific position weights based on the chronological order of interactions in each learner-specific relation sequence. The basic intuition is that the nodes interacted with more recently are more representative of the learner's current interests.

For each learner-specific relation $r \in \{dnu, int, eng\}$, let the corresponding relation sequence be ordered chronologically as

$$L_r = [(1, t_1)_{x_1}, (1, t_2)_{x_2}, \dots, (1, t_{N_r})_{x_{N_r}}], \quad (3.31)$$

where x_i denotes the i -th interacted node under relation r , and N_r denotes the total number of interacted nodes in that relation sequence. We assign a position weight to each node according to its position in the sequence:

$$w_{pos,i}^r = \frac{i}{N_r - 1}, \quad (3.32)$$

where i is the position index in the chronological sequence and starts from 0. The nodes that have same timestamp under the same learner-specific relation will have same position index. In this way, a more recent interaction is assigned a higher position weight.

This design allows the learner model to emphasize recent behaviors under each learner-specific relation. Under the DNU relation, more recent DNU concepts are given stronger importance because they are more likely to reflect the learner's current difficulties. Under the interest relation, recently updated concept interests contribute more strongly because they are more representative of the learner's current interests. Under the engagement relation, recently updated courses are emphasized because they better reflect the learner's current learning focus. Therefore, position weights provide a simple but effective way to model the learner's short-term interests from relation-specific interaction sequences.

For the example in Table 3.1, the learner-specific relation sequences are given by Eqs. 3.4–

3.6. Accordingly, the position weights under the DNU relation are

$$w_{pos}^{dnu} = [0, 1], \quad (3.33)$$

since $L_{dnu} = [(1, t_6)_{Concept_1}, (1, t_7)_{Concept_3}]$.

Under the interest relation, the position weights are

$$w_{pos}^{int} = [0, 0.5, 0.5], \quad (3.34)$$

according to the chronological order in $L_{int} = [(1, t_5)_{Concept_2}, (1, t_8)_{Concept_1}, (1, t_8)_{Concept_3}]$.

Under the engagement relation, the position weights are

$$w_{pos}^{eng} = [0, 1], \quad (3.35)$$

according to $L_{eng} = [(1, t_1)_{Course_1}, (1, t_8)_{Course_2}]$.

Based on these relation-specific position weights, the learner model can preserve not only the long-term interests patterns captured by semantic correlation and temporal order, but also the short-term interests reflected by recent interactions. In the following, we combine the updated relation-specific embeddings with the corresponding position weights to generate the final learner representation.

3.2.2.4 Construct relation-specific weight matrices

Inspired by RRGCN [4], we introduce a fixed relation-specific weight matrix for each learner-specific relation, namely E_{dnu} , E_{int} , and E_{eng} . These matrices are used as simple relation-specific transformations for the interacted neighbors under different relations. To initialize them, we adopt the Glorot uniform initialization method [5]. After initialization, these matrices are kept fixed during the online modeling stage. This design keeps the model simple, reduces computational cost, and helps avoid overfitting on small-scale datasets [4].

$$E_{dnu} = \text{Glorot_uniform}((\text{weight_size}, \text{weight_size})) \quad (3.36)$$

$$E_{int} = \text{Glorot_uniform}((\text{weight_size}, \text{weight_size})) \quad (3.37)$$

$$E_{eng} = \text{Glorot_uniform}((\text{weight_size}, \text{weight_size})) \quad (3.38)$$

where `weight_size` for our case will be the size of embedding vectors of each node.

3.2.2.5 Generate the Learner Model Embedding

$$\begin{aligned}
e_L = \frac{1}{w_{sum}^{dnu} + w_{sum}^{int} + w_{sum}^{eng}} & \left[\frac{1}{w_{sum}^{dnu}} \left[E_{dnu} \sum_{c \in N_L^{dnu}} w_c^{dnu} e_c^{dnu} \right] + \right. \\
& \frac{1}{w_{sum}^{int}} \left[E_{int} \sum_{c \in N_L^{int}} w_c^{int} e_c^{int} \right] + \\
& \left. \frac{1}{w_{sum}^{eng}} \left[E_{eng} \sum_{course \in N_L^{eng}} w_{course}^{eng} e_{course}^{eng} \right] \right] \quad (3.39)
\end{aligned}$$

where,

- e_L denotes the final learner model embedding by integrating the (L, DNU, Concept), (L, INTERESTED_IN, Concept) and (L, ENGAGED_IN, Course) relations.
- E_{dnu} , E_{int} , and E_{eng} denote the relation-specific weight matrices for the *dnu*, *int*, and *eng* relations, respectively.
- N_L^{dnu} , N_L^{int} , and N_L^{eng} denote the sets of interacted nodes of learner L under the *dnu*, *int*, and *eng* relations, respectively. Specifically, N_L^{dnu} and N_L^{int} contain concept nodes, while N_L^{eng} contains course nodes.
- e_c^{dnu} , e_c^{int} , and e_{course}^{eng} denote the updated embeddings of node c under the *dnu*, *int*, and *eng* relations, respectively. These embeddings already incorporate the results of the previous long-term interests modeling step.
- w_c^{dnu} , w_c^{int} , and w_{course}^{eng} denote the final weights of node under the corresponding learner-specific relation. These weights measure the importance of each interacted node when constructing the learner representation.
- For the DNU relation, the final weight of concept c is defined as

$$w_c^{dnu} = \begin{cases} w_{c,cos}^{dnu}, & P_L^{dnu} = 1 \\ \frac{1}{2} (w_{c,cos}^{dnu} + w_{c,pos}^{dnu}), & \text{otherwise,} \end{cases} \quad (3.40)$$

where $w_{c,cos}^{dnu}$ denotes the cosine similarity between the DNU concept and the learning material it belongs to:

$$w_{c,cos}^{dnu} = \cos(e_c, e_{lm}), \quad (3.41)$$

where e_c denotes the embedding of concept c , and e_{lm} denotes the embedding of the learning material.

- For the interest relation, the final weight of concept c is defined as

$$w_c^{int} = \begin{cases} w_{c,score}^{int}, & P_L^{int} = 1 \\ \frac{1}{2} (w_{c,score}^{int} + w_{c,pos}^{int}), & \text{otherwise,} \end{cases} \quad (3.42)$$

where $w_{c,score}^{int}$ denotes the interest score of concept c under the interest relation.

- For the engagement relation, the final weight of course $course$ is defined as

$$w_{course}^{eng} = \begin{cases} w_{course,eng}^{eng}, & P_L^{eng} = 1 \\ \frac{1}{2} (w_{course,eng}^{eng} + w_{course,pos}^{eng}), & \text{otherwise,} \end{cases} \quad (3.43)$$

where $w_{course,eng}^{eng}$ denotes the engagement level weight of course $course$ under the engagement relation. In our setting, the engagement level is mapped into a scalar value as follows:

$$w_{course,eng}^{eng} = \begin{cases} 0.1, & \text{if the engagement level is low,} \\ 0.3, & \text{if the engagement level is medium,} \\ 0.6, & \text{if the engagement level is high.} \end{cases} \quad (3.44)$$

where P_L^r denotes the number of distinct temporal positions under learner-specific relation r . If a relation contains only one interacted node, or if multiple interacted nodes share the same timestamp, then $P_L^r = 1$. In this case, only the relation-specific value weight is used.

- w_{sum}^{dnu} , w_{sum}^{int} , and w_{sum}^{eng} denote the sums of node weights under the DNU, interest, and engagement relations, respectively, i.e.,

$$w_{sum}^{dnu} = \sum_{c \in N_L^{dnu}} w_c^{dnu}, \quad w_{sum}^{int} = \sum_{c \in N_L^{int}} w_c^{int}, \quad w_{sum}^{eng} = \sum_{course \in N_L^{eng}} w_{course}^{eng}.$$

- $\sum_{c \in N_L^{dnu}} w_c^{dnu} e_c^{dnu}$, $\sum_{c \in N_L^{int}} w_c^{int} e_c^{int}$, and $\sum_{course \in N_L^{eng}} w_{course}^{eng} e_{course}^{eng}$ denote the weighted sums of node embeddings under the dnu , int , and eng relations, respectively.
- $\frac{1}{w_{sum}^{dnu}}$, $\frac{1}{w_{sum}^{int}}$, and $\frac{1}{w_{sum}^{eng}}$ are relation-wise normalization factors, which normalize the weighted aggregation results within each learner-specific relation.
- $\frac{1}{w_{sum}^{dnu} + w_{sum}^{int} + w_{sum}^{eng}}$ is the global normalization factor across the three learner-specific relations.
- Overall, Eq. 3.39 first aggregates the interacted nodes within each learner-specific relation, then applies a relation-specific transformation, and finally combines the three relation-aware representations to generate the final learner model embedding.

3.2.3 Recommendation

Based on the generated learner model embedding, we compute the cosine similarity between the learner embedding and the embeddings of the courses that the learner has not enrolled in CourseMapper, together with all courses available in the MOOC repository. The top 20 courses with the highest similarity scores are then selected as the recommended learning path for the learner.

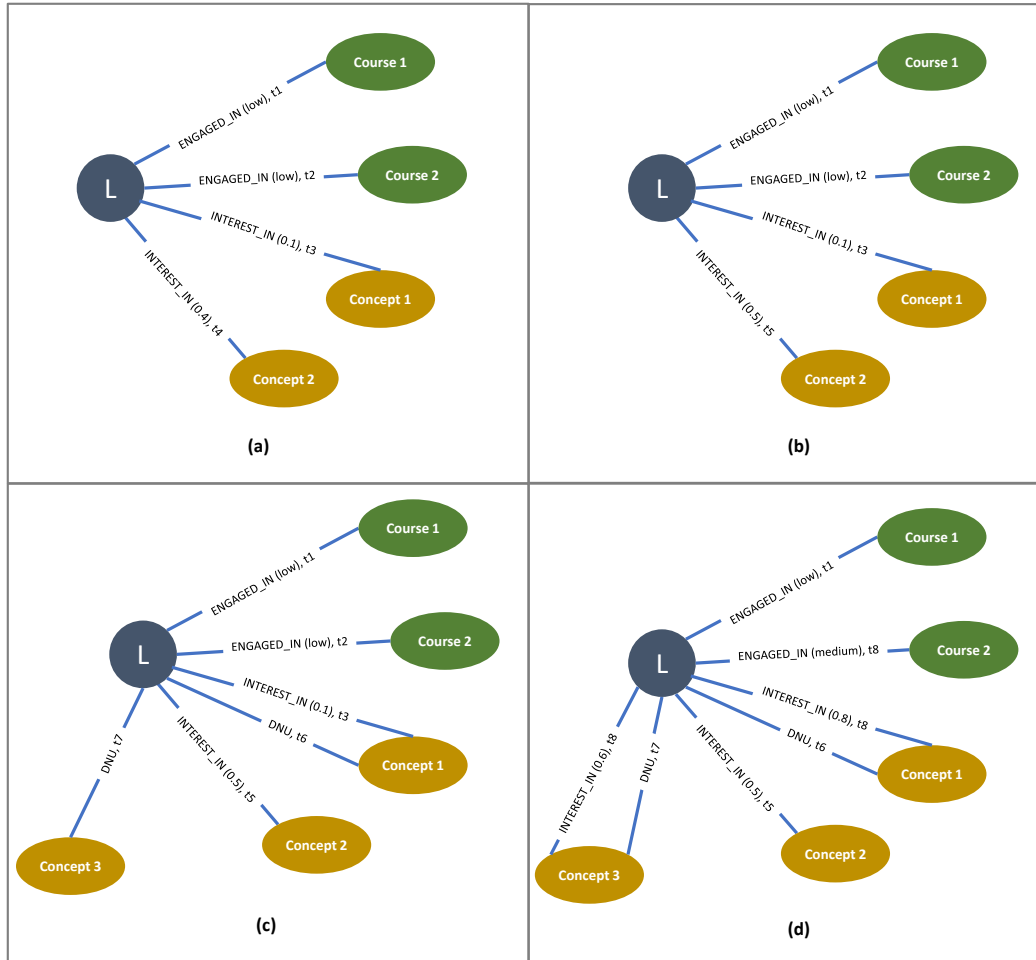


Figure 3.4: An example of historical interactions under multiple learner-specific relations. (a) shows the PKG before the first automatic recalculation. (b) shows the PKG after the first automatic recalculation. (c) shows the PKG before the second automatic recalculation. (d) shows the PKG after the second automatic recalculation. The label on each edge indicates the relation type and its corresponding timestamp. For the DNU relation, the edge represents whether the concept is marked as not understood. For the interest relation, the edge also shows the learner's interest score for the concept, whereas for the engagement relation, the edge shows the learner's engagement level for the course.

4 Implementation

In this chapter, we describe the technology stack used in this work and present the implementation details of LM-GNN. Finally, we introduce the key phases of the evaluation procedure.

4.1 System Architecture & Technologies

In terms of the technical framework of the system, we continue to use the technical framework of ConceptGCN[alatrash2024conceptgcn], as shown in Fig. 4.1.

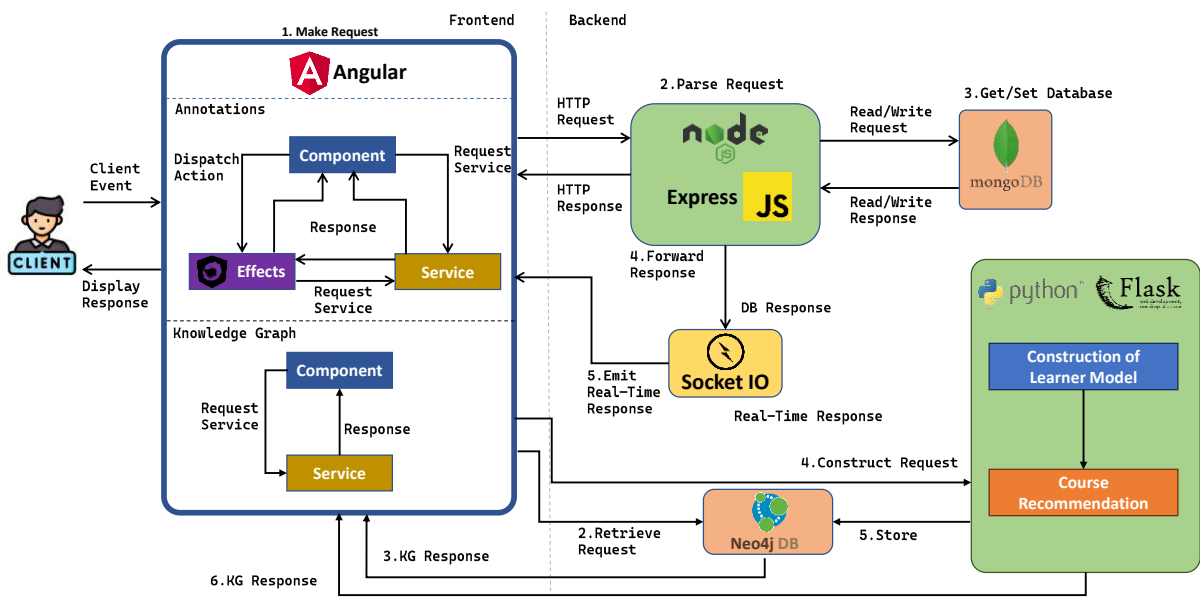


Figure 4.1: Technical architecture

The architecture is a distributed front-end and back-end separation system. The front-end Angular¹ realizes complex interactions through modular components. The back end uses

¹<https://angular.dev/overview>

Node.js² /Expressjs³ as the core gateway to interact with databases such as MongoDB⁴ and Neo4j⁵ and combines Python⁶ /Flask⁷ services to realize intelligent recommendation functions. In addition, Socket.IO⁸ enhances the ability for real-time interaction. The overall architecture has both high scalability and real-time performance, which is suitable for the implementation of knowledge recommendation and personalized systems.

4.2 Implementation of Course Embedding Construction

In both CourseMapper and MoocCentral, the construction of course embeddings follows a similar procedure. (1)course semantic embeddings are generated from course names using SBERT, (2)the course–concept mapping is identified, (3)the course embedding is computed using the course embedding equation 3.1 and 3.2.

The two systems differ, however, in the source of concept embeddings and in the method used to obtain the course–concept mapping. In CourseMapper, concept representations are not generated at this stage. Instead, the model directly uses the concept embeddings stored in CourseMapper Neo4j, namely, the embeddings produced by MR-GCN-RRGCN-based graph representation learning. Moreover, the mapping between courses and concepts cannot be obtained from CourseMapper Neo4j directly. Therefore, the system first queries MongoDB to retrieve the corresponding `course_id` of the learning materials and then uses this information to associate courses with concepts in CourseMapper Neo4j. By contrast, in MoocCentral, concept embeddings are generated explicitly from concept names using SBERT, and each course is directly connected to its related concepts through explicit edges in MoocCentral Neo4j. Consequently, the course–concept mapping can be obtained directly from the graph database.

To make the SBERT-based implementation pipeline in Neo4j explicit, the system first retrieves all Course nodes that do not yet have `name_embedding`, reads their course names, encodes these names using SBERT, and writes the resulting vectors back to Neo4j as `Course.name_embedding`. The system then retrieves all Concept nodes that do not yet have `name_embedding`, reads their concept names, encodes them using SBERT, and stores the resulting vectors as `Concept.name_embedding`. After the completion of these name-based embeddings, the system reads all `Course.name_embedding` values, all `Concept.name_embedding` values, and the course-to-concept mapping from Neo4j. For each course, the system collects all valid concept name embeddings associated with that course and computes the final course embedding as a weighted aggregation of the corresponding concept name embeddings, where each weight is defined by the cosine similarity between the course name embedding and the concept name embedding. If a course has no valid concept, the course embedding reduces to the course name embedding, , `Course.name_embedding`. Finally, the computed course embeddings are

² <https://nodejs.org/en>

³ <https://expressjs.com/>

⁴ <https://www.mongodb.com/>

⁵ <https://neo4j.com/>

⁶ <https://www.python.org/>

⁷ <https://flask.palletsprojects.com/en/stable/>

⁸ <https://socket.io/>

written back to Neo4j and stored as `Course.course_embedding`.

In the remainder of this section, we focus on three aspects of the implementation: (1) SBERT-based name embedding generation, (2) construction of the course–concept mapping, and (3) implementation of the course-embedding equation for the final course representation.

4.2.1 SBERT-based Name Embedding Generation

```

1 from ..database_connection.MoocCentral_connection import MoocCentralConnection
2 from sentence_transformers import SentenceTransformer
3
4 def generate_sbert_embeddings(self, name_dict):
5
6     if len(name_dict) == 0:
7         return {}
8     ids = list(name_dict.keys())
9     texts = list(name_dict.values())
10    embeddings = self.model.encode(
11        texts,
12        batch_size=64,
13        show_progress_bar=True,
14        convert_to_numpy=True
15    )
16
17    emb_dict = {}
18    for i, id_ in enumerate(ids):
19        emb_dict[id_] = embeddings[i]
20    return emb_dict

```

4.2.2 Construction of the Course–Concept Mapping

4.2.2.1 Course–Concept Mapping in MoocCentral

Because courses and concept nodes are directly connected in the MoocCentral database, the course–concept mapping can be obtained directly through database queries.

```

1 def get_course_concepts_mapping(self):
2
3     with self.driver.session() as session:
4
5         result = session.run("""
6             MATCH (course:Course)-[:CONTAINS_CONCEPT]->(concept:concept)
7             RETURN course.course_id as course_id, elementId(concept) as concept_id
8             """)
9
10        course_concept_mapping_dict = {}
11
12        for r in result:
13            course_id = r["course_id"]

```

```

14         concept_id = r["concept_id"]
15
16         if course_id not in course_concept_mapping_dict:
17             course_concept_mapping_dict[course_id] = []
18
19         course_concept_mapping_dict[course_id].append(concept_id)
20
21     return course_concept_mapping_dict

```

4.2.2.2 Course–Concept Mapping in CourseMapper

In CourseMapper, the construction of the course–concept mapping requires an additional synchronization step because the relationship between courses and concepts is not stored explicitly in CourseMapper Neo4j. Specifically, the system first identifies all LearningMaterial nodes that do not yet contain `course_id` and then queries MongoDB to retrieve the corresponding `courseId` for each material. The retrieved identifiers are subsequently written back to Neo4j, thereby enriching the LearningMaterial nodes with course-level information. After this synchronization process, the system extracts the course–concept mapping by traversing the LM_CONSISTS_OF relations between LearningMaterial nodes and Concept nodes and grouping the associated concepts according to `course_id`. This design enables CourseMapper to recover course-level concept associations from material-level links and provides the necessary mapping for subsequent course embedding construction.

```

1
2 def sync_course_id(self):
3
4     # Step 1: find which LearningMaterial has no course\_id LearningMaterial
5     with self.coursemapper_driver.session() as session:
6         result = session.run("""
7             MATCH (lm:LearningMaterial)
8             WHERE lm.course_id IS NULL
9             RETURN lm.mid AS mid
10            """)
11         mids = [record["mid"] for record in result]
12         print(f"Found {len(mids)} LearningMaterial without course_id in
13             CourseMapper Neo4j database.")
14
15     if not mids:
16         print("All LearningMaterial already have course_id.")
17         return
18
19     # MongoDB : search courseId
20     cursor = self.materials.find(
21         {"_id": {"$in": [ObjectId(str(mid)) for mid in mids]}},
22         {"_id": 1, "courseId": 1}
23     )
24     rows = []
25     for doc in cursor:
26         rows.append({
27             "mid": str(doc["_id"]),
28             "course_id": str(doc["courseId"])
29         })
30     print(f"Found {len(rows)} matching materials in MongoDB and write course_id
31         mapping to Neo4j.")

```

```

28         if not rows:
29             print("No matching materials found in MongoDB.")
30             return
31         # Step 3: write back to CourseMapper Neo4j
32         with self.coursemapper_driver.session() as session:
33             session.run("""
34                 UNWIND $rows AS row
35                 MATCH (lm:LearningMaterial {mid:row.mid})
36                 SET lm.course_id = row.course_id
37             """, rows=rows)
38         print(f"{len(rows)} course_id synced.")
39         #-----
40         # get course -> concept (only main concepts ) mapping
41         # assume the course_id has been synced to LearningMaterial nodes,
42         # and all concepts related to the course have rrgcn embeddings
43         #-----
44         def get_course_concepts_mapping(self):
45             with self.coursemapper_driver.session() as session:
46                 result = session.run(
47                     """
48                     MATCH (lm:LearningMaterial)-[:LM_CONSISTS_OF]->(c:Concept)
49                     WHERE lm.course_id IS NOT NULL
50                     RETURN lm.course_id AS course_id, c.cid AS concept_id
51                     """)
52                 course_concept_mapping_dict = {}
53                 for record in result:
54                     course_id = record["course_id"]
55                     concept_id = record["concept_id"]
56                     if course_id not in course_concept_mapping_dict:
57                         course_concept_mapping_dict[course_id] = []
58                     course_concept_mapping_dict[course_id].append(concept_id)
59         return course_concept_mapping_dict

```

4.2.3 Course Embedding Construction

By following the equation3.1 and 3.2. Course embedding can be calculated as following.

```

1 def compute_course_embedding(self):
2     # Load required data
3     course_name_emb_dict = self.get_course_name_embeddings()
4     concept_rrgcn_emb_dict = self.get_concept_rrgcn_embeddings()
5     course_concepts_mapping_dict = self.get_course_concepts_mapping()
6     course_emb_dict = {}
7     # Compute embedding for each course
8     for course_id, course_name_emb in course_name_emb_dict.items():
9         concept_ids_list = course_concepts_mapping_dict.get(course_id, [])
10        # Collect valid concept embeddings
11        valid_concept_rrgcn_emb_list = []
12        for concept_id in concept_ids_list:
13            if concept_id in concept_rrgcn_emb_dict:
14                valid_concept_rrgcn_emb_list.append(concept_rrgcn_emb_dict[concept_id])
15        # Fall back to course name embedding if no valid concept exists
16        if len(valid_concept_rrgcn_emb_list) == 0:
17            course_emb_dict[course_id] = course_name_emb

```

```

18         continue
19     # Stack concept embeddings into a matrix
20     concept_rrgcn_matrix = np.vstack(valid_concept_rrgcn_emb_list)
21     # Compute cosine-similarity weights
22     weight_list = []
23     for concept_rrgcn_emb in concept_rrgcn_matrix:
24         dot_product = np.dot(course_name_emb, concept_rrgcn_emb)
25         course_name_emb_norm = np.linalg.norm(course_name_emb)
26         concept_rrgcn_emb_norm = np.linalg.norm(concept_rrgcn_emb)
27         weight = dot_product / (course_name_emb_norm * concept_rrgcn_emb_norm)
28         weight_list.append(weight)
29     # Normalize weighted sum
30     weights_emb = np.array(weight_list)
31     weight_sum = np.sum(weights_emb)
32     weighted_emb_sum = (weights_emb[:, np.newaxis] * concept_rrgcn_matrix).sum(
33         axis=0)
34     course_emb = weighted_emb_sum / weight_sum
35     # Store final course embedding
36     course_emb_dict[course_id] = course_emb
37     return course_emb_dict

```

4.3 Implementation of Learner Model Embedding Construction

The construction of the learner model embedding follows a four-step pipeline. (1)for each user, the system extracts useful information under three relation types, namely `dnu`, `INTERESTED_IN`, and `ENGAGED_IN`, and organizes the retrieved records into relation-specific dictionaries. (2)based on each relation dictionary, the system constructs a correlation matrix, a mask matrix, and a sequential matrix to update the node embeddings under that relation. (3)the updated dictionaries are used to compute three relation components for each relation type, namely, weight summation and weighted embedding sum under each relationship. (4) the learner model embedding is computed according to the overall formulation^{3.39} by combining the three relation components.

4.3.1 Construction of Relation Dictionaries

The first step is implemented in `relationship_info.py`. The goal of this step is to construct a relation-specific dictionary for each user.

The relation dictionary has the following structure:

```

{
    cid: {
        "relationship": str,

```

```

        "updated_embedding": numpy_array,
        "first_weight_component": float,
        "timestamp": datetime,
        "position_weight": float,
        "position_time": int
    }
}

```

Here, `unupdated_embedding` denotes the original node embedding retrieved from the database. The field `first_weight_component` depends on the relation type. For `dnu`, it is the weight stored in the `dnu` edge. For `INTERESTED_IN`, it is the value of `interestScore`. For `ENGAGED_IN`, it is derived from the engagement level by using a predefined mapping. The timestamp source is also relation-dependent. For `dnu`, timestamps are retrieved from the user's `conceptTimestamps` field in MongoDB. For `INTERESTED_IN` and `ENGAGED_IN`, timestamps are read directly from Neo4j.

The following code snippet shows the core logic used to construct the `dnu` dictionary:

```

1 class DNUInfo(RelationshipBase):
2     def get_dnu_info(self):
3         dnu_info = {}
4         concept_timestamps = {}
5         with self.coursemapper_connection.get_session() as session:
6             result = session.run(
7                 """
8                 MATCH (u:User {uid:$uid})-[r:dnu]->(c:Concept)
9                 RETURN
10                    c.cid AS cid,
11                    c.mid AS mid,
12                    c.final_embedding AS concept_rrgc_embedding,
13                    r.weight AS first_weight_component
14                """,
15                uid=self.uid
16            )
17            for record in result:
18                if str(record["mid"]) not in self.valid_material_ids:
19                    continue
20                cid = str(record["cid"])
21                embedding = self.string_to_array(record["concept_rrgc_embedding"])
22                if embedding is None:
23                    continue
24                dnu_info[cid] = {
25                    "relationship": "dnu",
26                    "unupdated_embedding": embedding,
27                    "first_weight_component": float(record["first_weight_component"]),
28                    "timestamp": None
29                }
30            if not dnu_info:
31                return {}
32            user_collection = self.mongodb_connection.get_collection("users")
33            user_doc = user_collection.find_one({"_id": ObjectId(self.uid)})
34            if not user_doc:
35                return {}
36            timestamps_obj = user_doc.get("conceptTimestamps", {}) if user_doc else {}
37            for cid in dnu_info.keys():

```

```

38         matched = timestamps_obj.get(cid)
39         if matched is None:
40             raise ValueError(f"Missing timestamp for DNU concept: {cid}")
41         concept_timestamps[cid] = matched
42         dnu_info[cid]["timestamp"] = matched
43     dnu_info = self.compute_time_features(concept_timestamps, dnu_info)
44     return dnu_info

```

The following code snippet shows how the system constructs the INTERESTED_IN and ENGAGED_IN dictionaries:

```

1 class InterestInfo(RelationshipBase):
2     def get_interest_info(self):
3         interest_info = {}
4         concept_timestamps = {}
5         with self.coursemapper_connection.get_session() as session:
6             result = session.run("""
7             MATCH (u:User {uid:$uid})-[r:INTERESTED_IN]->(c:Concept)
8             RETURN
9             c.cid AS cid,
10            c.mid AS mid,
11            c.final_embedding AS concept_rrgcn_embedding,
12            r.interestScore AS first_weight_component,
13            r.updatedAt AS timestamp
14            """, uid=self.uid)
15            # Ignore records whose materials no longer exist in MongoDB.
16            for record in result:
17                if str(record["mid"]) not in self.valid_material_ids:
18                    continue
19                cid = str(record["cid"])
20                timestamp = record["timestamp"]
21                if timestamp is None:
22                    raise ValueError(f"Missing timestamp for INTERESTED_IN concept:
23                                   {cid}")
24                embedding = self.string_to_array(record["concept_rrgcn_embedding"])
25                if embedding is None:
26                    continue
27                interest_info[cid] = {
28                    "relationship": "INTERESTED_IN",
29                    "unupdated_embedding": embedding,
30                    "first_weight_component": float(record["first_weight_component"]
31                                                    )},
32                "timestamp": timestamp
33            }
34            concept_timestamps[cid] = timestamp
35            interest_info = self.compute_time_features(concept_timestamps, interest_info
36            )
37            return interest_info
38
39 class EngagementInfo(RelationshipBase):
40     LEVEL_MAP = {
41         "low": 0.1,
42         "medium": 0.3,
43         "high": 0.6
44     }
45     def get_engagement_info(self):

```

```

44     engagement_info = {}
45     course_timestamps = {}
46     with self.coursemapper_connection.get_session() as session:
47         result = session.run(
48             """
49             MATCH (u:User {uid:$uid})-[r:ENGAGED_IN]->(c:Course)
50             RETURN
51                 c.cid AS cid,
52                 c.course_embedding AS course_embedding,
53                 r.level AS level,
54                 r.timestamp AS timestamp
55             """,
56             uid=self.uid
57         )
58     for record in result:
59         cid = str(record["cid"])
60         # Check whether the course still exists in MongoDB.
61         if cid not in self.valid_course_ids:
62             continue
63         # Map engagement level to the first weight component.
64         first_weight_component = self.LEVEL_MAP.get(record["level"], 0.0)
65         timestamp = record["timestamp"]
66         if timestamp is None:
67             raise ValueError(f"Missing timestamp for ENGAGED_IN course: {cid}")
68         # Convert the embedding string to a numpy array.
69         embedding = self.string_to_array(record["course_embedding"])
70         if embedding is None:
71             continue
72         engagement_info[cid] = {
73             "relationship": "ENGAGED_IN",
74             "unupdated_embedding": embedding,
75             "first_weight_component": first_weight_component,
76             "timestamp": timestamp
77         }
78         # Store timestamps for temporal feature computation.
79         course_timestamps[cid] = timestamp
80     if not engagement_info:
81         return {}
82     engagement_info = self.compute_time_features(course_timestamps,
83                                                  engagement_info)
84     return engagement_info

```

4.3.2 Computation of Temporal Indicators

After the initial relation dictionary is constructed, the system computes two temporal indicators, namely `position_weight` and `position_time`. The timestamps are first deduplicated and sorted in chronological order from earlier to later interactions. Each unique timestamp is then assigned a group index starting from 0. This group index is stored as `position_time`. Based on the same grouping result, `position_weight` is computed as

$$\text{position_weight} = \frac{\text{group_id}}{\text{concept_numbers} - 1}. \quad (4.1)$$

Therefore, `position_time` records the temporal order of each timestamp group, whereas `position_weight` provides a normalized temporal signal. If only one node is retained in the relation dictionary, both values are set to 0. After that, the dictionary is sorted according to `position_time`.

The following function computes `position_weight` and `position_time`:

```

1 def compute_time_features(self, node_timestamps, info_dict):
2     # node_timestamps is a dictionary where the key is cid
3     # and the value is timestamp.
4     if any(ts is None for ts in node_timestamps.values()):
5         raise ValueError("Found None timestamp in node_timestamps")
6     concept_numbers = len(node_timestamps)
7     if concept_numbers == 1:
8         for cid in info_dict:
9             info_dict[cid]["position_weight"] = 0
10            info_dict[cid]["position_time"] = 0
11        return info_dict
12    # The same timestamp is grouped together.
13    # group_id starts from 0 and timestamps are sorted from earlier to later.
14    unique_timestamps = sorted(set(node_timestamps.values()))
15    timestamp_group = {ts: i for i, ts in enumerate(unique_timestamps)}
16    for cid, ts in node_timestamps.items():
17        group_id = timestamp_group[ts]
18        position_weight = (
19            group_id / (concept_numbers - 1)
20            if concept_numbers > 1
21            else 0.0
22        )
23        info_dict[cid]["position_weight"] = position_weight
24        info_dict[cid]["position_time"] = group_id
25    info_dict = dict(sorted(info_dict.items(), key=lambda x: x[1]["position_time"]))
26    return info_dict

```

4.3.3 Updating Node Embeddings Within Each Relation

The second step is implemented in `updated_embeddings.py`. In this step, the system updates node embeddings within each relation. The input is the relation dictionary constructed in the previous step. The system first reads `unupdated_embedding` from the dictionary and stacks the vectors into an embedding matrix. It then computes a correlation matrix based on cosine similarity, constructs a mask matrix according to `position_time`, and combines them into a sequential matrix. Finally, the sequential matrix is used to update the node embeddings, and the resulting vectors are written back to the dictionary as `updated_embedding`.

After this step, the dictionary is extended as follows:

```

{
    cid: {
        "relationship": str,
        "unupdated_embedding": numpy_array,
        "first_weight_component": float,
        "timestamp": datetime,

```



```

        "position_weight": float,
        "position_time": int,
        "updated_embedding": numpy_array
    }
}

```

The following code snippet shows the core implementation:

```

1      # Step 1: Build the embedding matrix from unupdated embeddings.
2      embeddings = [relation_info[cid]["unupdated_embedding"] for cid in cids]
3      embedding_matrix = np.vstack(embeddings)
4
5      # Step 2: Construct the correlation matrix by cosine similarity.
6      weight_matrix = np.zeros((num_nodes, num_nodes))
7
8      for i in range(num_nodes):
9          for j in range(num_nodes):
10             if i == j:
11                 weight_matrix[i][j] = 1.0
12             else:
13                 dot = np.dot(embeddings[i], embeddings[j])
14                 norm_i = np.linalg.norm(embeddings[i])
15                 norm_j = np.linalg.norm(embeddings[j])
16                 if norm_i == 0 or norm_j == 0:
17                     weight_matrix[i][j] = 0.0
18                 else:
19                     weight_matrix[i][j] = dot / (norm_i * norm_j)
20
21     # Step 3: Construct the mask matrix from temporal order.
22     positions = [relation_info[cid]["position_time"] for cid in cids]
23     mask_matrix = np.zeros((num_nodes, num_nodes))
24
25     for i in range(num_nodes):
26         for j in range(num_nodes):
27             if positions[i] <= positions[j]:
28                 mask_matrix[i][j] = 0
29             else:
30                 mask_matrix[i][j] = -10
31
32     # Step 4: Construct the sequential matrix.
33     sequential_matrix = np.where(mask_matrix == -10, 0, weight_matrix +
34                                 mask_matrix)
35
36     # Step 5: Update embeddings.
37     updated_matrix = np.dot(sequential_matrix, embedding_matrix)
38
39     # Step 6: Write the updated embeddings back to the dictionary.
40     for idx, cid in enumerate(cids):
41         relation_info[cid]["updated_embedding"] = updated_matrix[idx]

```

4.3.4 Computation of Relation Components

The third step is implemented in `relation_component.py`. In this step, the system computes one relation component for each relation type from the updated dictionary. More specifically, the system reads `first_weight_component`, `position_weight`, `position_time`, and `updated_embedding` from the dictionary. It then computes the node weight for each retained node and accumulates the weighted embedding sum together with the total relation weight.

The relation component has the following structure:

```
{
    "relationship_name": {
        "relationship_weight_sum": float,
        "weighted_embedding_sum": numpy_array
    }
}
```

If all retained nodes have `position_time` equal to 0, the implementation ignores `position_weight` and directly uses `first_weight_component` as the node weight. Otherwise, the node weight is computed as the average of `first_weight_component` and `position_weight`.

The following code snippet shows the core implementation:

```
1 class RelationComponent:
2     def compute_relation_component(self, relation_info):
3         if not relation_info:
4             return {}
5         cids = list(relation_info.keys())
6         relationship_name = relation_info[cids[0]]["relationship"]
7         # Check whether all position_time values are zero.
8         # If so, ignore position_weight and only use first_weight_component.
9         position_time = [relation_info[c]["position_time"] for c in cids]
10        ignore_position = all(ti == 0 for ti in position_time)
11        relation_weight_sum = 0.0
12        weighted_embedding_sum = None
13        for cid in cids:
14            concept = relation_info[cid]
15            first_weight_component = concept["first_weight_component"]
16            position_weight = concept["position_weight"]
17            node_embedding = concept["updated_embedding"]
18            if ignore_position:
19                node_weight = first_weight_component
20            else:
21                node_weight = (first_weight_component + position_weight) / 2
22            if weighted_embedding_sum is None:
23                weighted_embedding_sum = np.zeros_like(node_embedding)
24            weighted_embedding_sum += node_weight * node_embedding
25            relation_weight_sum += node_weight
26        relation_component = {
27            "relationship_name": {
28                "relationship_weight_sum": relation_weight_sum,
29                "weighted_embedding_sum": weighted_embedding_sum
```

```

30         }
31     }
32     return relation_component

```

4.3.5 Construction of the Final Learner Model Embedding

The final step is implemented in `learner_model.py`. In this step, the system first calls the previous three modules to obtain the three relation dictionaries, the updated dictionaries, and the corresponding relation components. It then generates three relation-specific transformation matrices, namely W_{rel_dnu} , $W_{rel_interest}$, and $W_{rel_engagement}$. Finally, the learner model embedding is computed by combining the three relation components according to the overall formulation. If the user has no valid interaction under any of the three relation types, the system returns a zero vector. Otherwise, the resulting learner model embedding is written back to Neo4j as the `learner_model_embedding` property of the corresponding User node.

The following code snippet shows the overall pipeline:

```

1
2 class LearnerModel:
3     def learner_model_pipeline(self):
4         """
5         1. Get relationship info dict for dnu, interested_in, engaged_in
6         2. Update node embeddings for each relationship
7         3. Compute relation component for each relationship
8         4. Generate random weight matrices for each relationship
9         5. Compute final learner model embedding
10        """
11        dnu_info_instance = DNUInfo(self.uid, self.coursemapper_connection, self.
12                                     mongodb_connection)
13        dnu_dict = dnu_info_instance.get_dnu_info()
14        interest_info_instance = InterestInfo(self.uid, self.coursemapper_connection
15                                              , self.mongodb_connection)
16        interest_dict = interest_info_instance.get_interest_info()
17        engagement_info_instance = EngagementInfo(self.uid, self.
18                                                  coursemapper_connection, self.mongodb_connection)
19        engagement_dict = engagement_info_instance.get_engagement_info()
20        embedding_updater = ConceptEmbeddingUpdater()
21        updated_dnu_dict = embedding_updater.update_embeddings(dnu_dict)
22        updated_interest_dict = embedding_updater.update_embeddings(interest_dict)
23        updated_engagement_dict = embedding_updater.update_embeddings(
24            engagement_dict)
25        relation_component_calculator = RelationComponent()
26        dnu_relation_component = relation_component_calculator.
27            compute_relation_component(updated_dnu_dict)
28        interest_relation_component = relation_component_calculator.
29            compute_relation_component(updated_interest_dict)
30        engagement_relation_component = relation_component_calculator.
31            compute_relation_component(updated_engagement_dict)
32        W_rel_dnu = glorot_seed((EMBEDDING_DIM, EMBEDDING_DIM)).numpy()
33        W_rel_interest = glorot_seed((EMBEDDING_DIM, EMBEDDING_DIM)).numpy()
34        W_rel_engagement = glorot_seed((EMBEDDING_DIM, EMBEDDING_DIM)).numpy()

```

The following code snippet shows the final computation of the learner model embedding:

```

1      has_dnu_relationship = bool(dnu_relation_component and "dnu" in
2                                  dnu_relation_component)
3      has_interest_relationship = bool(interest_relation_component and "
4                                       INTERESTED_IN" in interest_relation_component)
5      has_engagement_relationship = bool(engagement_relation_component and "
6                                       ENGAGED_IN" in engagement_relation_component)
7
8      dnu_weight_sum = (
9          dnu_relation_component["dnu"]["relationship_weight_sum"]
10         if has_dnu_relationship else None
11     )
12     interest_weight_sum = (
13         interest_relation_component["INTERESTED_IN"]["relationship_weight_sum"]
14         if has_interest_relationship else None
15     )
16     engagement_weight_sum = (
17         engagement_relation_component["ENGAGED_IN"]["relationship_weight_sum"]
18         if has_engagement_relationship else None
19     )
20
21     w_sum = 0
22
23     if has_dnu_relationship:
24         w_sum += dnu_weight_sum
25     if has_interest_relationship:
26         w_sum += interest_weight_sum
27     if has_engagement_relationship:
28         w_sum += engagement_weight_sum
29
30     # If the user has no interactions, return a zero vector.
31     if w_sum == 0:
32         return np.zeros(EMBEDDING_DIM)
33
34     dnu_term = (
35         (1 / dnu_weight_sum)
36         * W_rel_dnu.dot(dnu_relation_component["dnu"]["weighted_embedding_sum"])
37         if (has_dnu_relationship and dnu_weight_sum != 0) else np.zeros(
38             EMBEDDING_DIM)
39     )
40
41     interest_term = (
42         (1 / interest_weight_sum)
43         * W_rel_interest.dot(interest_relation_component["INTERESTED_IN"]["
44                               weighted_embedding_sum"])
45         if (has_interest_relationship and interest_weight_sum != 0) else np.
46             zeros(EMBEDDING_DIM)
47     )
48
49     engagement_term = (
50         (1 / engagement_weight_sum)
51         * W_rel_engagement.dot(engagement_relation_component["ENGAGED_IN"]["
52                               weighted_embedding_sum"])
53         if (has_engagement_relationship and engagement_weight_sum != 0) else np.
54             zeros(EMBEDDING_DIM)
55     )
56
57     learner_model_embedding = (dnu_term + interest_term + engagement_term) /

```

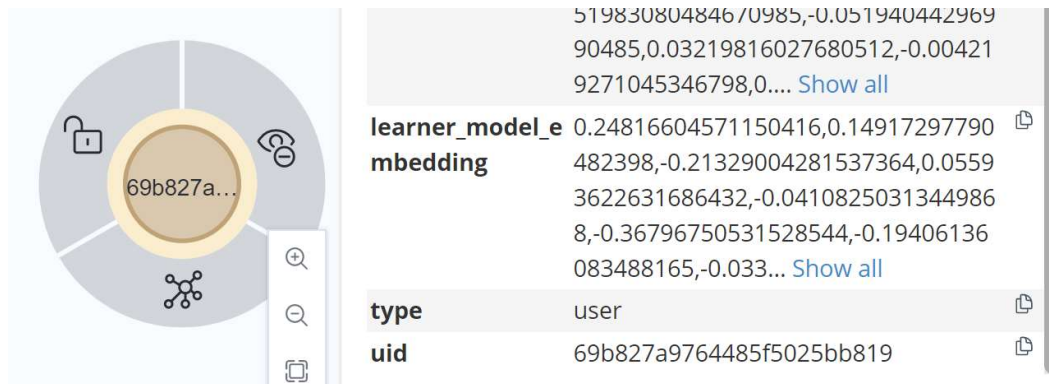


Figure 4.2: learner model embedding stored in neo4j.

```

50         w_sum
51         return learner_model_embedding

```

Finally, the computed learner model embedding is written back to Neo4j:

```

def store_learner_model_embedding(self, learner_model_embedding):
    """
    Store the learner model embedding back to Neo4j.
    """
    with self.coursemapper_connection.get_session() as session:
        session.run("""
        MATCH (u:User {uid:$uid})
        SET u.learner_model_embedding = $embedding
        """, uid=self.uid, embedding=",".join(map(str, learner_model_embedding)))

```

Shown in Fig.4.2 after these precess, the learner model embedding is stored in CourseMapper Neo4j.

4.4 Implementation of the Variant-based Recommendation Pipeline

In this section, we introduce how to process the MoocCube dataset, construct course embeddings and learner model embeddings, generate the top-k recommendation list for each user,

and calculate HR@K and NDCG@K.

Both concept name embeddings and course name embeddings are constructed using SBERT. The course embedding equation is the same as that used in the previous course embedding implementation.

In this section, we mainly focus on constructing the relationship dictionary and developing different variants based on the MoocCube dataset.

4.4.1 Offline Preprocessing and Embedding Preparation

File `preprocess.py` performs the main preprocessing step. It cleans raw user, course, and concept data, builds the course–concept mapping, filters invalid concepts, sorts user course records chronologically, and splits each user’s learning history into train and test sets. The resulting files, namely `processed_user.json`, `processed_course.json`, `processed_concept.json`, provide the cleaned foundation for all later stages.

File `concept_name_embedding.py` uses SBERT to encode concept names and stores both the concept-name embedding matrix and the `concept_id` → row index mapping. File `course_initial.py` first encodes course names with SBERT and then computes the initial course embedding as a weighted average of the corresponding concept-name embeddings. The weight is defined by the cosine similarity between the course-name embedding and the concept-name embedding. If a course has no valid concept, its initial embedding falls back to its course-name embedding.

File `learner_training_data.py` constructs the learner-side training data. The training courses are taken directly from the user’s train split and sorted by time. The training concepts are generated indirectly from the user’s training courses. For each training course, the file selects the Top-20 concepts according to the cosine similarity between the course initial embedding and the concept-name embeddings. The selected concepts are then merged across all training courses, deduplicated, and stored together with their source-course information and temporal order.

Files `learner_course_final.py` and `learner_concept_final.py` construct user-specific final embeddings for courses and concepts, respectively. For each user, these files build a correlation matrix, a time-mask matrix, and a sequential matrix, and then update the corresponding course or concept embeddings by matrix multiplication. Because these files follow the same matrix-update logic as the original model, we do not repeat the full implementation details here. Their outputs are later used by the relation-dictionary files and the variant-construction file.

4.4.2 Construction of the ENROLL Relation Dictionary

The first core step is implemented in `enroll_relationship_dict.py`. The goal of this file is to construct the ENROLL relation dictionary for one user. The file first identifies the user’s training courses and computes a learner initial embedding by averaging the initial embeddings of these training courses. It then reads the user-specific course-final embeddings, assigns temporal

indicators, computes node-level weights, and finally accumulates relation-level statistics. The resulting dictionary is later used as the ENROLL-side input to the variant-construction file.

The ENROLL relation dictionary stores both relation-level statistics and node-level metadata. Its structure is shown below.

```
user_enroll_relation_dict = {
    "ENROLL": {
        "summary": {
            "relation_type": "ENROLL",
            "node_count": 3,
            "ignore_position": False,
            "relation_weight_sum": 2.1458,
            "has_enroll_relationship": True
        },
        "nodes": {
            "course_id_1": {
                "course_id": "course_id_1",
                "enroll_time": "2017-08-16 10:32:24",
                "position_time": 0,
                "position_weight": 0.0,
                "first_weight_component": 0.8123,
                "node_weight": 0.40615,
                "course_final_embedding_index": 12,
                "course_initial_embedding_index": 45
            }
        }
    }
}
```

This dictionary is designed so that later computation can directly read both node-level and relation-level information. Specifically, the `summary` block stores the relation type, the number of retained nodes, whether temporal position should be ignored, the total relation weight, and whether the ENROLL relation exists. The `nodes` block stores, for each course node, the temporal order, the temporal weight, the first weight component, the final node weight, and the indices required to retrieve both the course final embedding and the course initial embedding. In later steps, the model reads `node_weight`, `first_weight_component`, and the embedding indices from this dictionary to compute relation components and final learner embeddings.

The learner initial embedding is computed by averaging the initial embeddings of the user's training courses:

```
1
2 learner_course_vectors = []
3 for item in training_courses:
4     course_id = str(item.get("course_id", "")).strip()
5     if course_id in course_initial_id_to_index:
6         idx = int(course_initial_id_to_index[course_id])
7         learner_course_vectors.append(course_initial_embeddings[idx])
8
```

```

9 if len(learner_course_vectors) > 0:
10     learner_initial_embedding = np.mean(np.array(learner_course_vectors, dtype=np.
        float32), axis=0)
11 else:
12     learner_initial_embedding = None
13 \end{verbatim}
14
15 After that, the file constructs the node block, computes \texttt{position\_time} and
    \texttt{position\_weight}, and then computes \texttt{first\_weight\_component},
    \texttt{node\_weight}, and \texttt{relation\_weight\_sum}:
16
17 \begin{verbatim}
18 nodes[course_id] = {
19     "course_id": course_id,
20     "enroll_time": item.get("enroll_time"),
21     "position_time": 0,
22     "position_weight": 0.0,
23     "first_weight_component": None,
24     "node_weight": None,
25     "course_final_embedding_index": int(item.get("course_final_embedding_index")),
26     "course_initial_embedding_index": int(course_initial_id_to_index[course_id]),
27 }
28
29 if node_count == 1 or len(set(enroll_times)) == 1:
30     ignore_position = True
31     for course_id in nodes:
32         nodes[course_id]["position_time"] = 0
33         nodes[course_id]["position_weight"] = 0.0
34 else:
35     ignore_position = False
36     unique_times = sorted(set(enroll_times), key=parse_time)
37     time_to_position = {t: i for i, t in enumerate(unique_times)}
38     for course_id in nodes:
39         t = nodes[course_id]["enroll_time"]
40         position_time = time_to_position[t]
41         nodes[course_id]["position_time"] = position_time
42         nodes[course_id]["position_weight"] = position_time / (node_count - 1)
43
44 for course_id in nodes:
45     initial_idx = nodes[course_id]["course_initial_embedding_index"]
46     course_initial_embedding = course_initial_embeddings[initial_idx]
47
48     if learner_initial_embedding is None:
49         first_weight_component = 0.0
50     else:
51         first_weight_component = cosine_similarity(learner_initial_embedding,
            course_initial_embedding)
52
53     nodes[course_id]["first_weight_component"] = float(first_weight_component)
54
55     if ignore_position:
56         node_weight = first_weight_component
57     else:
58         node_weight = (first_weight_component + nodes[course_id]["position_weight"])
            / 2
59     nodes[course_id]["node_weight"] = float(node_weight)
60     relation_weight_sum += float(node_weight)

```


4.4.3 Construction of the INTEREST Relation Dictionary

The second core step is implemented in `interest_relationship_dict_11.py`. This file follows almost the same pattern as the ENROLL relation file, but it operates on the concept side instead of the course side. Therefore, we do not repeat the full construction process here. The main difference is that each concept node additionally stores the source course from which the concept was generated, the concept-name embedding index, and the source-course initial-embedding index.

The node structure in the INTEREST relation dictionary is shown below.

```
nodes[concept_id] = {
    "concept_id": concept_id,
    "interest_time": item.get("interest_time"),
    "source_course_id": source_course_id,
    "position_time": 0,
    "position_weight": 0.0,
    "first_weight_component": None,
    "node_weight": None,
    "concept_final_embedding_index": int(item.get("concept_final_embedding_index")),
    "concept_name_embedding_index": int(concept_name_id_to_index[concept_id]),
    "source_course_initial_embedding_index": int(course_initial_id_to_index[source_course_id])
}
```

The weight update logic is also analogous to that of the ENROLL relation. The key difference is that the first weight component is defined as the cosine similarity between the concept-name embedding and the source-course initial embedding:

```
1
2 for concept_id in nodes:
3     concept_idx = nodes[concept_id]["concept_name_embedding_index"]
4     course_idx = nodes[concept_id]["source_course_initial_embedding_index"]
5
6     concept_name_embedding = concept_name_embeddings[concept_idx]
7     source_course_initial_embedding = course_initial_embeddings[course_idx]
8
9     first_weight_component = cosine_similarity(
10         concept_name_embedding,
11         source_course_initial_embedding,
12     )
13     nodes[concept_id]["first_weight_component"] = float(first_weight_component)
14
15     if ignore_position:
16         node_weight = first_weight_component
17     else:
18         node_weight = (first_weight_component + nodes[concept_id]["position_weight"]
19                        ) / 2
20
21     nodes[concept_id]["node_weight"] = float(node_weight)
22     relation_weight_sum += float(node_weight)
```

The resulting dictionary again contains a `summary` block and a `nodes` block. In the later variant-construction step, the model reads this dictionary in exactly the same way as it reads the ENROLL dictionary.

4.4.4 Construction of Learner-Embedding Variants

The most important stage is implemented in `variant.py`. This file takes the ENROLL and INTEREST relation dictionaries and converts them into final learner embeddings under different variant settings. Its design has two layers. First, it constructs reusable relation components from the two relation dictionaries. Second, it selects different component vectors and normalization strategies according to the variant configuration table.

For each relation, the file first constructs a reusable component dictionary:

```

1
2 components = {
3     "exists": summary["node_count"] > 0,
4     "node_count": int(summary["node_count"]),
5     "ignore_position": bool(summary["ignore_position"]),
6     "relation_weight_sum": float(summary["relation_weight_sum"]),
7     "direct_sum": np.zeros(EMBEDDING_DIM, dtype=np.float32),
8     "node_weight_sum": np.zeros(EMBEDDING_DIM, dtype=np.float32),
9     "first_weight_sum": np.zeros(EMBEDDING_DIM, dtype=np.float32),
10    "position_weight_sum": np.zeros(EMBEDDING_DIM, dtype=np.float32),
11 }
12
13 for node_id in nodes:
14     info = nodes[node_id]
15     emb_idx = int(info[final_index_key])
16     emb = final_embeddings[emb_idx].astype(np.float32)
17
18     components["direct_sum"] += emb
19     components["node_weight_sum"] += float(info["node_weight"]) * emb
20     components["first_weight_sum"] += float(info["first_weight_component"]) * emb
21     components["position_weight_sum"] += float(info["position_weight"]) * emb

```

This design is important because all later variants are built from the same reusable relation components. The field `direct_sum` is the unweighted sum of final node embeddings. The field `node_weight_sum` is the weighted sum using `node_weight`. The field `first_weight_sum` uses only `first_weight_component`. The field `position_weight_sum` uses only `position_weight`. The field `relation_weight_sum` is used later for relation-level normalization. If `ignore_position=True`, the positional contribution degenerates and the positional variant no longer reflects temporal differentiation.

After the relation components have been prepared, the file uses `VARIANT_TABLE` to define seven learner-embedding variants:

```

1
2 VARIANT_TABLE = {
3     1: {"component": "node_weight_sum", "use_random_matrix": True, "
4         use_relation_normalization": True, "use_outer_normalization": True},
5     2: {"component": "node_weight_sum", "use_random_matrix": False, "
6         use_relation_normalization": True, "use_outer_normalization": True},

```

```

5     3: {"component": "direct_sum", "use_random_matrix": True, "
      use_relation_normalization": True, "use_outer_normalization": True},
6     4: {"component": "first_weight_sum", "use_random_matrix": True, "
      use_relation_normalization": True, "use_outer_normalization": True},
7     5: {"component": "position_weight_sum", "use_random_matrix": True, "
      use_relation_normalization": True, "use_outer_normalization": True},
8     6: {"component": "node_weight_sum", "use_random_matrix": True, "
      use_relation_normalization": False, "use_outer_normalization": False},
9     7: {"component": "node_weight_sum", "use_random_matrix": False, "
      use_relation_normalization": False, "use_outer_normalization": False},
10  }

```

This table controls three switches. The first switch selects which component vector is used, namely `direct_sum`, `node_weight_sum`, `first_weight_sum`, or `position_weight_sum`. The second switch determines whether a random matrix is applied to the selected relation term. The third switch determines whether normalization is applied after combining ENROLL and INTEREST. Therefore, Variant 1 is the full version with weighted components, random matrices, relation-level normalization, and outer normalization. Variant 2 removes the random matrix. Variant 3 replaces the weighted relation component with the direct sum. Variant 4 uses only the first weight component. Variant 5 uses only the positional component. Variant 6 removes both relation-level normalization and outer normalization while still keeping the random matrix. Variant 7 removes both normalization and the random matrix, thus producing the simplest weighted variant.

For one relation, the file first builds one relation term according to the selected settings:

```

1
2 def build_one_relation_term(relation_components, matrix, setting):
3     if not relation_components["exists"]:
4         return np.zeros(EMBEDDING_DIM, dtype=np.float32)
5
6     term = choose_component_vector(relation_components, setting["component"]).copy()
7
8     if setting["use_relation_normalization"]:
9         relation_weight_sum = relation_components["relation_weight_sum"]
10        if relation_weight_sum != 0:
11            term = term / relation_weight_sum
12
13    if setting["use_random_matrix"]:
14        term = matrix.dot(term)
15
16    return term.astype(np.float32)

```

This function first chooses the component vector, then optionally divides it by `relation_weight_sum`, and finally optionally applies the random matrix. Therefore, the same relation dictionary can generate different relation terms under different variant settings.

After the ENROLL term and the INTEREST term have been obtained, the file combines them into the final learner embedding:

```

1 def combine_two_relation_terms(enroll_term, interest_term, enroll_components,
2   interest_components, setting):
3     enroll_exists = enroll_components["exists"]
4     interest_exists = interest_components["exists"]
5
6     if (not enroll_exists) and (not interest_exists):

```

```

6         return np.zeros(EMBEDDING_DIM, dtype=np.float32)
7     if enroll_exists and (not interest_exists):
8         return enroll_term.astype(np.float32)
9     if interest_exists and (not enroll_exists):
10        return interest_term.astype(np.float32)
11
12    final_embedding = enroll_term + interest_term
13
14    if setting["use_outer_normalization"]:
15        total_weight_sum = (
16            enroll_components["relation_weight_sum"] +
17            interest_components["relation_weight_sum"]
18        )
19        if total_weight_sum != 0:
20            final_embedding = final_embedding / total_weight_sum
21
22    return final_embedding.astype(np.float32)

```

This combination step is also essential. If only one relation exists, the learner embedding is simply the corresponding relation term. If both relations exist, the two relation terms are added. If outer normalization is enabled, the combined vector is further divided by the total relation weight of ENROLL and INTEREST. The helper function `build_all_variants_for_one_user(user_id)` repeats this procedure for all seven settings and returns all variant embeddings for the current user.

4.4.5 Negative Sampling and Recommendation Generation

Files `negative_sample.py` and `recommendation_list.py` generate candidate sets and recommendation lists, so we keep this section brief.

File `negative_sample.py` treats each user's test courses as positive items and defines as negatives all unlearned courses. For each positive course, it samples a fixed number of negatives and merges them into the final candidate set.

File `recommendation_list.py` ranks the candidate set. It retrieves the learner embedding of the selected variant, builds the candidate course matrix, computes scores by matrix multiplication, sorts them in descending order, and returns the Top- K course identifiers.

4.4.6 Evaluation Metrics

The final stage is implemented in `evaluation.py`. This file evaluates each variant by comparing the recommendation list with the user's ground-truth test courses. Three metrics are reported, namely HR@K at the ground-truth level, HR@K at the user level, and NDCG@K following the equations 5.15–5.16, 5.18.

The corresponding implementation is shown below.

```

1
2 def compute_hr_ground_truth_level(recommendation_dict, ground_truth_dict, k):

```

```

3     total_hits = 0
4     total_ground_truth = 0
5     for user_id, gt_items in ground_truth_dict.items():
6         rec_items = recommendation_dict.get(user_id, {}).get("top20_course_ids", [])
7         total_hits += len(set(rec_items) & set(gt_items))
8         total_ground_truth += len(gt_items)
9     return 0.0 if total_ground_truth == 0 else float(total_hits / total_ground_truth)
10
11 def compute_hr_user_level(recommendation_dict, ground_truth_dict, k):
12     hit_user_count = 0
13     user_count = 0
14     for user_id, gt_items in ground_truth_dict.items():
15         rec_items = recommendation_dict.get(user_id, {}).get("top20_course_ids", [])
16         if len(gt_items) == 0:
17             continue
18         user_count += 1
19         if len(set(rec_items) & set(gt_items)) > 0:
20             hit_user_count += 1
21     return 0.0 if user_count == 0 else float(hit_user_count / user_count)
22
23 def compute_ndcg(recommendation_dict, ground_truth_dict, k):
24     total_ndcg = 0.0
25     user_count = 0
26     for user_id, gt_items in ground_truth_dict.items():
27         if len(gt_items) == 0:
28             continue
29         rec_items = recommendation_dict.get(user_id, {}).get("top20_course_ids", [])
30         gt_set = set(gt_items)
31
32         dcg = 0.0
33         for rank, course_id in enumerate(rec_items, start=1):
34             rel = 1 if course_id in gt_set else 0
35             dcg += (2 ** rel - 1) / math.log2(rank + 1)
36
37         ideal_hits = min(len(gt_items), k)
38         idcg = 0.0
39         for rank in range(1, ideal_hits + 1):
40             idcg += 1.0 / math.log2(rank + 1)
41
42         if idcg == 0:
43             continue
44
45         total_ndcg += dcg / idcg
46         user_count += 1
47
48     return 0.0 if user_count == 0 else float(total_ndcg / user_count)

```

In summary, the whole recommendation framework starts from offline preprocessing and embedding preparation, then constructs ENROLL and INTEREST relation dictionaries, builds multiple learner-embedding variants from these dictionaries, generates candidate sets, ranks candidate courses, and finally evaluates recommendation quality with HR@K and NDCG@K. Among all files, the methodological core lies in the relation-dictionary construction of File 10

and File 11 and in the variant-construction mechanism of File 12, because these files determine how user history is organized, weighted, and transformed into the final learner representation used for recommendation.

5 Evaluation

5.1 MOOCCube Dataset

We conduct experiments on a real-world large-scale dataset, MOOCCube¹, which was collected from the XuetangX² platform[26]. The MOOCCube dataset contains 55,203 users and 706 courses. Each user is enrolled in at least four courses, and the average number of enrolled courses per user is 6.42. Detailed statistical information is reported in Table 5.1. The entity types in the MOOCCube dataset are listed in Table 5.2. The MOOCCube dataset contains the following relation types: concept–field, concept–paper, course–concept, course–video, parent–son (taxonomy), prerequisite–dependency, school–course, school–teacher, teacher–course, user–course, user–video, and video–concept.

5.2 Experimental Settings

The dataset is described in Section 5.1. In this section, we describe the data processing procedure used for evaluation.

In the MOOCCube dataset, each user has an enrollment history, it is a chronological sequence of enrolled courses. We split the enrolled course sequence of each user into training and test subsets according to time. Specifically, we use the courses enrolled from October 1, 2016 to December 30, 2017 as training data, and those enrolled from January 1, 2018 to March 31, 2018 as test data [28, 15, 6].

For evaluation, the test set of each user consists of the courses enrolled from January 1, 2018 to March 31, 2018. Each course in the test set is treated as a positive sample. Because ranking all items for every user during evaluation is computationally expensive [10], for each positive sample, we randomly sample 99 courses that the user has never enrolled in as negative samples [6, 15]. We then compute the recommendation score for each candidate course, including both positive and negative samples, by measuring the cosine similarity between the candidate course embedding and the constructed learner model embedding. The top- k courses with the highest scores are returned as the recommendation list for each user. The courses in the test set are treated as the ground-truth items.

¹ <http://moocdata.cn/data/MOCCube>

² <https://www.xuetangx.com/>

Table 5.1: Statistics of the MOOCCube dataset

Dataset	#Courses	#Users	#Average Interactions
MOOCCube	706	55,203	6.42

Table 5.2: Entities in the MOOCCube Dataset

Entity type	Prefix of ID	Important fields	File
concept	K_	name, en, explanation	concept.json
course	C_	name, about, core_id, video_order, video_name, chapter	course.json
paper	P_	title, author, venue, abstract, year, num_citation, ...	paper.json
school	S_	name, about	school.json
teacher	T_	name, about	teacher.json
user	U_	name, course_order, enroll_time	user.json
video	V_	name, duration, start, end, text	video.json
taxonomy	K_T_	name	concept.json

In terms of dataset statistics, the original dataset contains 55,203 users and 706 courses, and each user is enrolled in at least four courses. After splitting the enrolled course sequence of each user into training and test subsets, we remove users with an empty training set or an empty test set. The final dataset contains 9,617 users.

5.3 Personal Knowledge Graph Construction in MOOCCube

We first examine the MOOCCube dataset and identify all relations that are directly connected to users. Based on these user-related relations, we construct an initial PKG, as shown in Fig.5.1. This initial PKG is a coarse graph that includes the relations (User, *enrolled*, Course), (User, *watched*, Video), and (User, *interested*, Concept).

We then analyze this initial PKG to determine whether it can support the construction of our learner model embedding. Our learner model is designed to capture both long-term and short-term interests. Therefore, temporal information is required for each relation in order to compute the position weight of each connected node and to construct relation-specific sequential matrices for updating node embeddings. However, the initial PKG does not fully satisfy this requirement, because not all user-related relations in MOOCCube provide usable temporal information.

Among the relations in the initial PKG, the relation (User, *enrolled*, Course) can be used. In MOOCCube, each user is associated with a chronological sequence of enrolled courses, and each enrolled course has a corresponding enrollment time. Therefore, the timestamp of each (User, *enrolled*, Course) edge can be directly defined as the enrollment time of the corre-

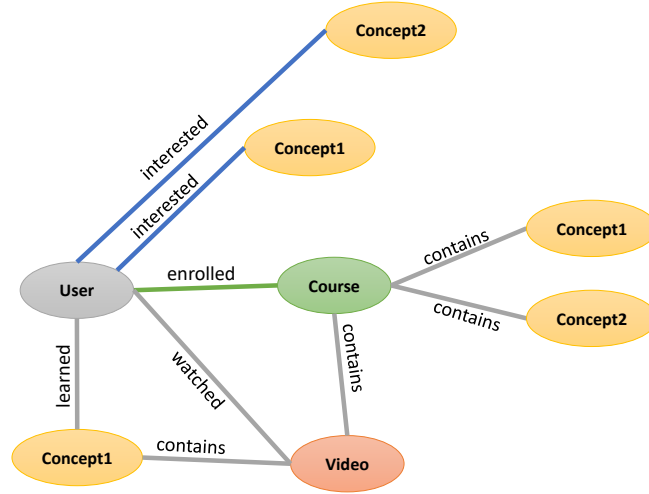


Figure 5.1: The initial PKG in MOOCCube.

sponding course. In contrast, the relation (User, *watched*, Video) cannot be used in our learner model. As shown in Table 5.2, the video file records only the video name, duration, and the transcript of each video, where each transcript is segmented into sentences with corresponding start and end times. These start and end times indicate the positions of sentences within the video transcript rather than the time at which a user watched the video. Moreover, the user–video relation file records only user ID and video ID pairs and does not provide any interaction timestamps. Therefore, the relation (User, *watched*, Video) does not contain the temporal information required by our learner model. Based on this relation, we can further derive the relation (User, *learned*, Concept) by assuming that if a user has watched a video, then the user has learned all concepts covered in that video. However, because no useful temporal information can be obtained from the watched-video relation, no meaningful temporal information can be assigned to the learned-concept relation either. Therefore, this relation is also unsuitable for our temporal learner model.

Based on this analysis, we construct a second PKG, shown in Fig. 5.2, which is the final PKG used to build the learner model embedding. In this final PKG, we retain the enrollment relation and further derive a concept-level relation from course enrollment. Specifically, when a user enrolls in a course, we assume that the user is interested in the concepts covered by that course. This allows us to construct edges of the form (User, *interested*, Concept) based on course enrollment rather than video-watching behavior. For each such edge, we assign the enrollment time of the corresponding course as its timestamp. If the same concept appears in multiple courses enrolled by the same user, we assign the most recent enrollment time among the courses containing that concept as the timestamp of that concept. In this way, we provide temporal information for both user–course edges and user–concept edges in the final PKG.

Because the number of concepts in MOOCCube is very large, directly using all concepts results in extremely high computational cost. In practice, when all concepts are included, the

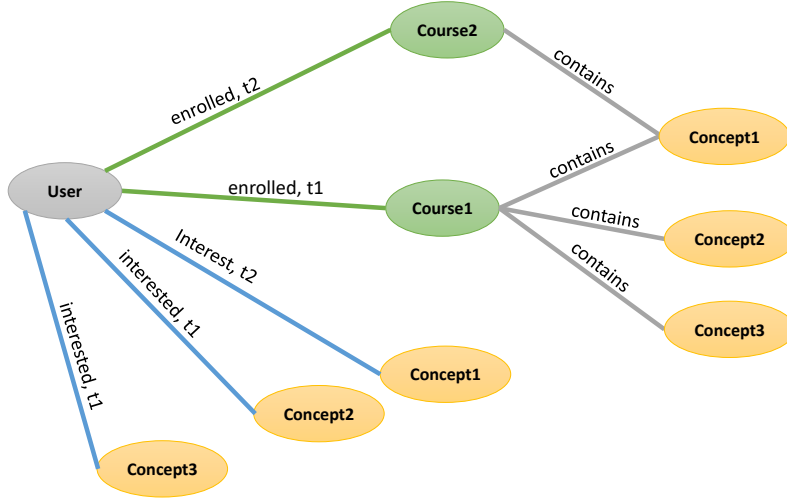


Figure 5.2: The final PKG in MOOCCube.

computer crashes during training. Therefore, we further restrict the concept set used in training. For each course in the training set of each user, we select the first 20 associated concepts. The final PKG is constructed from the training courses and the selected concepts, and it is used to build the learner model embedding for MOOCCube. This final PKG also serves as the basis for the subsequent ablation studies.

5.4 Ablation Study

To conduct an ablation study on the MOOCCube dataset using our LM-GNN method, we first generalize the learner model equation to ensure compatibility with all personal knowledge graphs. Then we construct learner model equation for MOOCCube.

5.4.1 Generalized Learner Model Equation

The generalized learner model equation is given by:

$$e_L = \frac{1}{w_{sum}^R} \sum_{r \in R} \frac{1}{w_{sum}^r} \left[E_r \sum_{c \in N_L^r} \omega_c^r e_c^r \right] \quad (5.1)$$

Components.

- e_L denotes the embedding of the learner model.

- R denotes the set of relation types.
- $r \in R$ denotes a specific relation.
- N_L^r denotes the set of nodes connected to learner L under relation r .
- E_r denotes the relation-specific weight matrix for relation r , which is initialized using the Glorot method.
- w_c^r denotes the weight of node c under relation r .
- e_c^r denotes the embedding of node c under relation r .
- $w_{sum}^r = \sum_{c \in N_L^r} w_c^r$ denotes the sum of node weights under relation r .
- $w_{sum}^R = \sum_{r \in R} w_{sum}^r$ denotes the sum of relation-level weights over all relations.

5.4.2 Learner model Equation for MoocCube

$$e_L = \frac{1}{w_{sum}^{enrolled} + w_{sum}^{interested}} \left[\frac{1}{w_{sum}^{enrolled}} \left[E_{enrolled} \sum_{course \in N_L^{enrolled}} w_{course}^{enrolled} e_{course}^{enrolled} \right] + \frac{1}{w_{sum}^{interested}} \left[E_{interested} \sum_{c \in N_L^{interested}} w_c^{interested} e_c^{interested} \right] \right] \quad (5.2)$$

where,

$$w_{sum}^{enrolled} = \sum_{course \in N_L^{enrolled}} w_{course}^{enrolled}, \quad w_{sum}^{interested} = \sum_{c \in N_L^{interested}} w_c^{interested}, \quad (5.3)$$

- e_L denotes the final learner model embedding by integrating the (User, *enrolled*, Course) and (User, *interested*, Concept) relations.
- $E_{enrolled}$, $E_{interested}$ denote the relation-specific weight matrices for the (User, *enrolled*, Course) and (User, *interested*, Concept) relations, respectively.
- $N_L^{enrolled}$, $N_L^{interested}$ denote the sets of interacted nodes of learner L under the (User, *enrolled*, Course) and (User, *interested*, Concept) relations, respectively. Specifically, $N_L^{enrolled}$ contain course nodes, while $N_L^{interested}$ contains concept nodes.
- $e_{course}^{enrolled}$, $e_c^{interested}$ denote the updated embeddings of node c under the (User, *enrolled*, Course) and (User, *interested*, Concept) relations, respectively. These embeddings should be updated following the equation 3.7
- $w_{course}^{enrolled}$, $w_c^{interested}$ denote the final weights of node c under the corresponding learner-specific relation. These weights measure the importance of each interacted node when constructing the learner representation.
- For the (User, *enrolled*, Course) relation, the final weight of concept c is defined as

$$w_{course}^{enrolled} = \begin{cases} w_{course,cos}^{enrolled}, & P_L^{enrolled} = 1 \\ \frac{1}{2} (w_{course,cos}^{enrolled} + w_{course,pos}^{enrolled}), & \text{otherwise,} \end{cases} \quad (5.4)$$

where $w_{course,cos}^{enrolled}$ denotes the cosine similarity between the course initial embedding following equation 3.2 and the learner initial embedding.

$$w_{course,cos}^{enrolled} = \cos(e_{course}, e_L^{initial}), \text{ where, } e_L^{initial} = \frac{1}{|C_L^{enrolled}|} \sum_{course \in C_L^{enrolled}} e_{course} \quad (5.5)$$

- For the (User, *interested*, Concept) relation, the final weight of concept c is defined as

$$w_c^{interested} = \begin{cases} w_{c,cos}^{interested}, & P_L^{interested} = 1 \\ \frac{1}{2} (w_{c,cos}^{interested} + w_{c,pos}^{interested}), & \text{otherwise,} \end{cases} \quad (5.6)$$

where $w_{c,cos}^{interested}$ is the cosine similarity between concept and the course it belonging to. If one concept is belongs to multiple courses, we choose the course that has the same timestamp as this concept.

where P_L^r denotes the number of distinct temporal positions under learner-specific relation r . If a relation contains only one interacted node, or if multiple interacted nodes share the same timestamp, then $P_L^r = 1$. In this case, only the relation-specific value weight is used.

5.4.2.1 Special Case: If User Has No Interested Concept

After preprocessing the MoocCube dataset, we found that not every enrolled course is associated with concept nodes. Specifically, 15 users have one enrolled course and two concepts, and 2,187 users have one enrolled course and multiple concepts. By contrast, we did not observe users with multiple enrolled courses but zero concepts, and we also did not observe users with one enrolled course and one concept. Therefore, we explicitly consider the special case in which a learner has enrolled courses but no interested concepts. In this case, only the (User, *enrolled*, Course) relation is used.

$$e_L = \frac{1}{w_{sum}^{enrolled}} \left(E_{enrolled} \sum_{course \in C_L^{enrolled}} w_{course}^{enrolled} e_{course}^{enrolled} \right) \quad (5.7)$$

5.4.3 Variants based on the Generalized Learner model Equation

Variant 1: The entire generalized equation:

$$e_L = \frac{1}{w_{sum}^R} \sum_{r \in R} \frac{1}{w_{sum}^r} \left[E_r \sum_{c \in N_L^r} w_c^r e_c^r \right] \quad (5.8)$$

Variant 2: Remove specific-relation matrix:

$$e_L = \frac{1}{w_{sum}^R} \sum_{r \in R} \frac{1}{w_{sum}^r} \left[\sum_{c \in N_L^r} w_c^r e_c^r \right] \quad (5.9)$$

Variant 3: Remove node weight:

$$e_L = \frac{1}{w_{sum}^R} \sum_{r \in R} \frac{1}{w_{sum}^r} \left[E_r \sum_{c \in N_L^r} e_c^r \right] \quad (5.10)$$

Variant 4: Remove the position weight of each node:

$$e_L = \frac{1}{w_{sum}^R} \sum_{r \in R} \frac{1}{w_{sum}^r} \left[\sum_{c \in N_L^r} w_{c,cos}^r e_c^r \right] \quad (5.11)$$

$w_{c,cos}^r$ is the first component of a node weight under relationship r .

Variant 5: Remove the first component weight of each node:

$$e_L = \frac{1}{w_{sum}^R} \sum_{r \in R} \frac{1}{w_{sum}^r} \left[\sum_{c \in N_L^r} w_{c,pos}^r e_c^r \right] \quad (5.12)$$

$w_{c,pos}^r$ is the position weight of each node under relationship r .

Variant 6: Remove the normalization components:

$$e_L = \sum_{r \in R} \left[E_r \sum_{c \in N_L^r} w_c^r e_c^r \right] \quad (5.13)$$

Variant 7: Remove the normalization components and the specific weight matrix:

$$e_L = \sum_{r \in R} \left[\sum_{c \in N_L^r} w_c^r e_c^r \right] \quad (5.14)$$

5.5 Evaluation Metrics

We employ HR@K and NDCG@K to evaluate model performance[7, 9]. Both metrics have been widely used to evaluate top-K recommendation[14]. Specifically, HR@K is used in two complementary formulations, namely a ground-truth-level formulation [15] and a user-level formulation [6]. As described in [9], HR@K is a recall-oriented metric that measures whether the ground-truth item is successfully retrieved within the Top-K recommendation list. From the ground-truth-level perspective, HR@K measures the proportion of true relevant items that are

successfully retrieved, thereby reflecting the overall coverage of relevant items by the model. This formulation is particularly suitable for evaluation settings in which each user is associated with multiple ground-truth items, since it provides a finer-grained assessment of recall over the complete set of relevant items. From the user-level perspective, HR@K measures whether at least one relevant item is successfully recommended to each user within the Top-K list, thereby reflecting the recommendation success rate across users. This formulation is more closely aligned with the practical objective of providing each user with at least one correct recommendation, and it is particularly suitable for leave-one-out or single-positive evaluation settings because of its simplicity and interpretability. In contrast, NDCG@K further considers the ranking position of the ground-truth item and assigns higher scores when correctly recommended items appear at higher rank position[11]. Therefore, while HR@K evaluates whether the model successfully hits the target item, NDCG@K evaluates the quality of the ranking position of relevant items.

5.5.1 HR@K Equation 1 (Ground-Truth Level)

$$\text{HR@K} = \frac{\sum_{u=1}^U \text{Hits@K}_u}{|GT|} \quad (5.15)$$

In Eq. 5.15, K denotes the cutoff of the recommendation list, U denotes the number of users, $\text{Hits}_u@K$ denotes the number of ground-truth items of user u that appear in the top- K recommendation list, GT denotes the set of all ground-truth items in the test set, and $|GT|$ denotes the total number of ground-truth items[15, 11].

5.5.2 HR@K Equation 2 (User Level)

$$\text{HR@K} = \frac{1}{N} \sum_u I(|R_u \cap T_u|) \quad (5.16)$$

$$I(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0. \end{cases} \quad (5.17)$$

In Eq. 5.16, R_u denotes the final top- K recommendation list for user u , T_u denotes the test set of user u , N denotes the number of users in the test data, and $I(x)$ denotes the indicator function defined in Eq. 5.17. This formulation of HR@K differs from the ground-truth-level formulation in its focus. The ground-truth-level formulation counts the number of retrieved ground-truth items, whereas the user-level formulation determines whether there is at least one hit for each user[6].

5.5.3 NDCG@K Equation

$$\text{NDCG}@K = \frac{1}{U} \sum_{u=1}^U \frac{\text{DCG}_u@K}{\text{IDCG}_u@K} \quad (5.18)$$

$$\text{DCG}_u@K = \sum_{i=1}^K \frac{2^{\text{rel}_u^i} - 1}{\log_2(i + 1)} \quad (5.19)$$

In Eq. 5.18, $\text{DCG}_u@K$ denotes the discounted cumulative gain of user u , $\text{IDCG}_u@K$ denotes the ideal discounted cumulative gain of user u , rel_u^i denotes the relevance of the item at rank i for user u , and U denotes the number of users. In our experiments, we adopt binary relevance, where $\text{rel}_u^i = 1$ for a relevant item and $\text{rel}_u^i = 0$ for a non-relevant item.

5.5.4 A Simple Example of Metric Calculation

To illustrate the evaluation procedure more clearly, we provide a simple example. Assume that the course database contains 50 courses and 3 users. For each course in the test set of a user, we treat it as a positive item, , a ground-truth item. For each positive item, we sample 10 negative items from the courses in which the user has not enrolled. For brevity, the specific negative samples are omitted.

For each user, the model outputs a Top-5 recommendation list. The training data, test data, and the final Top-5 recommendation lists are shown in Table 5.3.

As shown in Table 5.3, the ground-truth item of User 1, namely $C4$, does not appear in the Top-5 recommendation list. For User 2, both ground-truth items appear in the Top-5 recommendation list, where $C16$ appears in the first position and $C17$ appears in the fifth position. For User 3, the ground-truth item $C35$ appears in the second position.

5.5.4.1 HR@5 at the Ground-Truth Level

At the ground-truth level, $\text{HR}@5$ measures the proportion of ground-truth items that appear in the Top-5 recommendation lists:

$$\text{HR}_{\text{item}}@5 = \frac{\sum_{u=1}^U |R_u^{(5)} \cap T_u|}{\sum_{u=1}^U |T_u|}, \quad (5.20)$$

where $R_u^{(5)}$ denotes the Top-5 recommendation list of user u , and T_u denotes the test set of user u .

In this example, $T_1 = \{C4\}$, $T_2 = \{C16, C17\}$, and $T_3 = \{C35\}$, so the total number of

Table 5.3: Training data, test data, and Top-5 recommendation lists

User	Training courses	Test courses	Top-5 recommendation list
User 1	$C1, C2, C3$	$C4$	1. $C8$ 2. $C12$ 3. $C20$ 4. $C27$ 5. $C41$
User 2	$C10, C11, C12, C13$	$C16, C17$	1. $C16$ 2. $C22$ 3. $C3$ 4. $C40$ 5. $C17$
User 3	$C33, C34$	$C35$	1. $C7$ 2. $C35$ 3. $C12$ 4. $C18$ 5. $C49$

ground-truth items is $|T_1| + |T_2| + |T_3| = 1 + 2 + 1 = 4$. The numbers of hits for the three users are $|R_1^{(5)} \cap T_1| = 0$, $|R_2^{(5)} \cap T_2| = 2$, and $|R_3^{(5)} \cap T_3| = 1$, respectively. Therefore,

$$HR_{\text{item}@5} = \frac{0 + 2 + 1}{4} = \frac{3}{4} = 0.75.$$

5.5.4.2 HR@5 at the User Level

At the user level, HR@5 measures the proportion of users for whom at least one ground-truth item appears in the Top-5 recommendation list:

$$HR_{\text{user}@5} = \frac{1}{U} \sum_{u=1}^U \mathbf{1}(|R_u^{(5)} \cap T_u| > 0), \quad (5.21)$$

where $\mathbf{1}(\cdot)$ is an indicator function.

In this example, the indicator values for Users 1, 2, and 3 are 0, 1, and 1, respectively. Hence,

$$HR_{\text{user}@5} = \frac{0 + 1 + 1}{3} = \frac{2}{3} \approx 0.6667.$$

5.5.4.3 NDCG@5

The dataset-level NDCG@5 is defined as

$$NDCG@5 = \frac{1}{U} \sum_{u=1}^U \frac{DCG_u@5}{IDCG_u@5}, \quad (5.22)$$

where

$$DCG_u@5 = \sum_{i=1}^5 \frac{2^{rel_{u,i}} - 1}{\log_2(i+1)}. \quad (5.23)$$

Here, binary relevance is adopted:

$$rel_{u,i} = \begin{cases} 1, & \text{if the item at rank } i \text{ is relevant,} \\ 0, & \text{otherwise.} \end{cases}$$

The discount values at ranks 1 to 5 are

$$\frac{1}{\log_2(2)} = 1.0000, \quad \frac{1}{\log_2(3)} \approx 0.6309, \quad \frac{1}{\log_2(4)} = 0.5000, \quad \frac{1}{\log_2(5)} \approx 0.4307, \quad \frac{1}{\log_2(6)} \approx 0.3869.$$

For User 1, the recommendation list $R_1^{(5)} = [C8, C12, C20, C27, C41]$ contains no relevant item from $T_1 = \{C4\}$. Therefore, $DCG_1@5 = 0$, $IDCG_1@5 = 1.0000$, and $NDCG_1@5 = 0$.

For User 2, the recommendation list $R_2^{(5)} = [C16, C22, C3, C40, C17]$ and the test set $T_2 = \{C16, C17\}$ yield the relevance sequence $[1, 0, 0, 0, 1]$. Thus,

$$DCG_2@5 = \frac{1}{\log_2(2)} + \frac{1}{\log_2(6)} = 1.0000 + 0.3869 = 1.3869.$$

Because User 2 has two relevant items, the ideal ranking is $[1, 1, 0, 0, 0]$, and

$$IDCG_2@5 = \frac{1}{\log_2(2)} + \frac{1}{\log_2(3)} = 1.0000 + 0.6309 = 1.6309.$$

Therefore,

$$NDCG_2@5 = \frac{1.3869}{1.6309} \approx 0.8504.$$

For User 3, the recommendation list $R_3^{(5)} = [C7, C35, C12, C18, C49]$ and the test set $T_3 = \{C35\}$ yield the relevance sequence $[0, 1, 0, 0, 0]$. Thus,

$$DCG_3@5 = \frac{1}{\log_2(3)} \approx 0.6309.$$

Because User 3 has one relevant item, $IDCG_3@5 = 1.0000$, and therefore

$$NDCG_3@5 = \frac{0.6309}{1.0000} = 0.6309.$$

Finally,

$$NDCG@5 = \frac{NDCG_1@5 + NDCG_2@5 + NDCG_3@5}{3} = \frac{0 + 0.8504 + 0.6309}{3} = \frac{1.4813}{3} \approx 0.4938.$$

Thus, in this example, $HR_{item}@5 = 0.75$, $HR_{user}@5 \approx 0.6667$, and $NDCG@5 \approx 0.4938$. These results indicate that ground-truth-level HR@5, user-level HR@5, and NDCG@5 capture different aspects of recommendation quality. Specifically, ground-truth-level HR@5 measures the proportion of retrieved ground-truth items, user-level HR@5 measures the proportion of users who receive at least one correct recommendation, and NDCG@5 further evaluates the ranking quality by assigning higher importance to relevant items that appear at higher positions in the recommendation list.

5.6 Baseline Methods

We compare LM-GNN with the following baseline methods. The results are shown in 5.4

DARL[15] is a reinforcement-learning-based hybrid course recommendation method for MOOCs, which combines hierarchical reinforcement learning with a dynamic attention mechanism to revise user profiles and model changing user preferences for sequential course recommendation.

ACKRec[6] is a supervised knowledge concept recommendation method for MOOCs, which uses a heterogeneous information network with meta-path-based graph convolution and attention to learn user/concept representations, and then applies matrix factorization for top-N knowledge concept recommendation.

AGNN[29] is a supervised sequential course recommendation method for MOOCs, which uses GCN, LSTM-based attention, CNN-derived item features, and BPR ranking to learn user and course embeddings from user-course interaction data.

KGAN[27] is a supervised course recommendation method for MOOCs, which uses a course knowledge graph with grouped propagation and hierarchical attention aggregation to model user preferences and predict course selection.

TP-GNN[22] is a supervised course recommendation method for MOOCs, which captures the high-level semantic relationships among courses by a graph convolutional network.

5.7 Discussion

5.7.1 Research Questions

Based on the design objective of LM-GNN and the characteristics of the MOOCCube dataset, this study addresses the following two research questions.

RQ1: Can LM-GNN, which is constructed on learner-specific relationships, jointly model learners' long-term interests and short-term interests under different relations and produce a learner representation for course recommendation?

RQ2: To what extent does the effectiveness of LM-GNN depend on the richness of learner-specific relationships and on the granularity of behavioral and temporal information? What are its applicability and limitations on a dataset such as MOOCCube, where available relations are limited and temporal signals are coarse-grained?

These two research questions correspond to the two main goals of this study. RQ1 examines whether the design of LM-GNN is conceptually valid, that is, whether learner-specific relationships can serve as the foundation of learner modeling and whether the sequential matrix and the position weight can model long-term interests and short-term interests, respectively. RQ2 examines the conditions under which the method is applicable, with particular attention to the extent to which this learner modeling framework depends on the richness of relation types, the reliability of behavioral signals, and the granularity of temporal information in the data.

5.7.2 Answer to RQ1

The results for RQ1 show that LM-GNN can construct learner representations based on learner-specific relationships and can jointly model long-term interests and short-term interests under different relations. More specifically, the model represents the structural relation between a learner and connected nodes under each learner-specific relation. To capture historical interests, the model introduces a sequential matrix that combines semantic relatedness among nodes with the temporal order of interactions. Through the correlation matrix, the model preserves semantic consistency among historical interactions. Through the mask matrix, it further constrains the temporal direction of influence, so that past interactions affect later representations, whereas later interactions do not affect earlier ones. The sequential matrix is therefore better suited to characterizing long-term interests that are gradually formed and persist over time.

To capture short-term interests, the model further introduces a relation-specific position weight. This mechanism assigns larger weights to more recent interactions according to their positions in the temporal sequence, thereby emphasizing the contribution of behaviors that are closer to the current time to the learner representation. Under this design, LM-GNN derives relation-specific representations from different learner-specific relationships and then integrates

them into a unified learner model embedding for subsequent course recommendation. From the perspective of method design, LM-GNN therefore answers how learner-specific relationships support the joint modeling of long-term interests and short-term interests, which shows that the learner model is conceptually well founded.

The experimental results also show that this joint modeling is only partially verified on MOOC-Cube. As reported in Table 5.4, Variant 5, which retains only the position component, performs the worst among all LM-GNN variants, whereas Variant 4, which removes the position component, performs very close to the full model in Variant 1. This result suggests that under the current data conditions, the short-term preference signal associated with the position weight does not fully take effect. A more precise answer to RQ1 is therefore that LM-GNN establishes a learner modeling framework that jointly models long-term interests and short-term interests at the methodological level, but on MOOC-Cube this joint modeling is reflected mainly in the modeling logic of long-term interests, whereas the advantage of short-term preference modeling is not fully realized.

5.7.3 Answer to RQ2

The results for RQ2 show that the effectiveness of LM-GNN strongly depends on the richness of learner-specific relationships and on the granularity of behavioral and temporal information. The original design of LM-GNN is not built around a single user–course relation. Instead, it emphasizes modeling learner behavior under multiple learner-specific relationships and then integrating the resulting representations across relations. The method is therefore better suited to platforms that provide multiple learner-specific relations together with finer-grained behavioral records.

MOOC-Cube has clear limitations in this regard. Although the dataset contains several user-related relations, such as user–course and user–video, our analysis shows that the only relation that provides usable temporal information for the learner model is (User, enrolled, Course). In addition, this study derives the relation (User, interested, Concept) from course enrollment and assigns the enrollment time of the corresponding course to the concept-level relation. However, this interested relation is not an independently observed behavior. It is an approximate relation inferred from enrollment. In other words, the learner-specific relationships that LM-GNN can finally use on MOOC-Cube are not only limited in number but also highly dependent on the enrollment relation for temporal information.

More importantly, the main temporal signal that can be directly used in MOOC-Cube is enrollment time, and enrollment time does not necessarily reflect a learner’s actual engagement or immediate change of interest. Enrolling in a course only indicates that the user completes the enrollment action. It does not show whether the user truly studies the course, nor whether the user continues to attend to the course or its related concepts in the recent period. As a result, using enrollment time directly as the basis of the position weight makes recent enrollment an approximation of current preference, which weakens the effectiveness of short-term preference modeling. The relatively low performance of LM-GNN on MOOC-Cube should therefore not be interpreted as evidence that the learner model design is invalid. Rather, it indicates that the method depends strongly on relation richness and temporal granularity. On platforms such as



Figure 5.3: The enrolled course count distribution on training set and testing set

CourseMapper, which provide richer learner-specific relationships and finer-grained behavioral timestamps, the design of LM-GNN is more likely to achieve its full potential.

5.7.4 Performance Analysis

5.7.4.1 Data sparsity in learner history

After temporal splitting, MOOCCube exhibits substantial sparsity in learner history. The final experimental set contains 9,617 users. The distribution in Fig.5.3 shows that more than 2,000 users have only one training course, and another more than 2,000 users have only two training courses. As a result, a substantial proportion of learner representations is constructed from highly limited historical information. This sparsity directly affects the performance of LM-GNN. LM-GNN constructs learner model embeddings from historical courses and concepts. When a user has only one or two training courses, the learner representation inevitably depends too heavily on a small number of courses and their related concepts, and therefore cannot fully reflect a more stable and comprehensive interest structure. If the ground-truth course in the test stage differs substantially from the subject area represented by the few training courses, the recommendation is likely to deviate. Sparsity in learner history is therefore an important factor that limits the performance of LM-GNN on MOOCCube.

5.7.4.2 Limited learner-specific relationships in MOOCCube

A key premise of LM-GNN is that learner-specific relationships are sufficiently rich to support multi-relation learner modeling from multiple behavioral perspectives. However, on MOOCCube, the learner-specific relations that can be used reliably are mainly the enrolled relation and the interested relation derived from it. Since the interested relation is not an independently observed behavior but is indirectly generated from the enrollment relation, its semantic source and temporal source are both highly coupled with enrollment. As a consequence, MOOCCube does not provide learner-specific relationships that are both rich enough and sufficiently independent to support a complete multi-relation learner modeling framework.

In other words, what is evaluated on MOOCCube is not the full capability of LM-GNN under ideal conditions, but a simplified version under restricted relations and restricted temporal signals. Therefore, when LM-GNN performs worse than supervised baselines on this dataset, a more reasonable interpretation is that MOOCCube does not fully support a learner model based on multiple learner-specific relationships, rather than that the design is conceptually flawed.

5.7.4.3 Enrollment as a weak preference signal

In MOOCCube, positive feedback mainly comes from enrollment behavior. However, enrollment itself is only a weak preference signal. A user may enroll in a course without engaging in sustained study afterward, and enrollment does not necessarily indicate that the course truly reflects the user's current direction of interest. On open course platforms in particular, users may enroll for browsing, trial, bookmarking, or random exploration, which introduces noise into enrollment behavior.

For LM-GNN, this issue is further amplified. The method does not treat enrollment as a simple standalone label. Instead, it derives concept relations from enrollment, constructs the sequential matrix on this basis, and then applies the position weight. If some enrollment actions are highly random, this noise not only enters the learner representation but may also propagate through sequential updates and temporal weighting. This weak preference nature of enrollment is therefore another important reason for the relatively low recommendation performance on MOOCCube.

This limitation also highlights an important difference between MOOCCube and CourseMapper. In CourseMapper, the learner model is built by jointly considering not only course selection but also the learner's level of course engagement. As a result, the behavioral signal used for learner modeling is more informative and more reliable than enrollment alone. Under such conditions, learner-specific relationships are better aligned with the learner's actual interests, which makes the resulting learner representation more stable and allows the model design to function more effectively.

5.7.4.4 Coarse-grained temporal signal weakens short-term preference modeling

The position weight in this study is designed to emphasize the contribution of recent behaviors to learner representation. Its core assumption is that interactions closer to the current time better reflect the learner's current needs and interests. For this design, the quality of temporal information is crucial. If timestamps truly record actual engagement with courses or concepts, larger weights on recent interactions are more likely to capture short-term interests accurately.

However, the main timestamp available in MOOCCube is enrollment time. This timestamp only records when course enrollment occurs. It does not indicate when the user actually watches the course, when the user engages continuously, or when the user returns to a concept. Therefore, recent enrollment on MOOCCube does not necessarily mean recent engagement, nor does it necessarily indicate the strongest current preference. Under such conditions, the position weight may emphasize not the most representative recent interest but recent noisy behavior.

The ablation results support this interpretation. As shown in Table 5.4, Variant 5, which keeps only the position component, performs the worst, whereas Variant 4, which removes the position component, performs close to the full model in Variant 1. This finding indicates that on MOOCCube, coarse-grained timestamps alone cannot support effective short-term preference modeling. The issue therefore does not lie in the idea of modeling short-term interests itself, but in the fact that the temporal information provided by MOOCCube is not fine-grained enough to support this design effectively.

5.7.4.5 Unsupervised learner modeling versus supervised ranking objectives

LM-GNN differs fundamentally from supervised baselines such as DARL, AGNN, KGAN, and TP-GNN in its training objective. LM-GNN focuses on constructing learner representations by combining learner-specific relationships, the sequential matrix, and the position weight into an interpretable learner model embedding. By contrast, supervised baselines usually optimize the ranking task directly and use negative samples, pairwise ranking loss, or other supervised objectives to learn which items should rank higher.

This difference directly affects the results under sampled Top- K recommendation. In the experimental setting of this study, the model ranks courses within a candidate set consisting of one positive sample and 99 negative samples. For supervised baselines, the training process explicitly uses both positive and negative signals to optimize the ranking boundary, which makes it easier to achieve higher HR@ K and NDCG@ K under this evaluation protocol. Prior work [16] also shows that the choice of negative sampling strategy is important in recommendation, because negative samples help the model learn not only what a user may prefer but also what the user is less likely to prefer. This ability is largely absent in LM-GNN. As an unsupervised learner modeling method, LM-GNN does not directly optimize the objective that positive samples should be ranked above negative samples. As a result, it is naturally weaker at distinguishing relevant courses from irrelevant ones. This is one important reason why LM-GNN

performs worse than supervised baselines under sampled Top- K evaluation.

5.7.4.6 Ablation findings: simpler variants are more robust on MOOCCube

Although the full version of LM-GNN performs weakly on MOOCCube, the ablation study still provides several valuable findings. First, Variant 2 and Variant 7 clearly outperform the full model in Variant 1. This result indicates that under the current data conditions, removing the relation-specific matrix leads to better recommendation performance. A reasonable explanation is that the learner-specific relationships available in MOOCCube are limited in number and insufficiently independent. Under such conditions, relation-specific transformation matrices do not necessarily learn stable and complementary relation semantics, and may instead introduce additional noise because the parameterization is too strong.

Second, Variant 2 clearly outperforms Variant 4 and Variant 5, each of which keeps only one component. This result shows that combining semantic relatedness with temporal position remains more effective than using either signal alone. This observation provides indirect support for the core idea of LM-GNN, namely that learner modeling should not rely on a single signal but should combine historical structural information with recent behavioral information. In other words, although the full model does not achieve the best performance on MOOCCube, the ablation study still shows that the key design idea of LM-GNN has a certain degree of validity, while under the current data conditions a simpler and more robust implementation works better than the full relation-specific parameterization.

5.7.5 Summary of Discussion

Overall, the experimental results do not support the claim that LM-GNN outperforms existing supervised baselines on MOOCCube, but this does not imply that the learner model design itself is invalid. Instead, the results show that LM-GNN, as a fine-grained learner modeling framework based on learner-specific relationships, depends strongly on the richness of relations and on the granularity of behavioral and temporal information. MOOCCube mainly provides enrollment-based coarse-grained signals. It lacks both independent and rich learner-specific relationships and reliable temporal support for short-term preference modeling. As a result, the full advantage of LM-GNN cannot be realized on this dataset.

At the same time, the ablation study further shows that under data conditions with weak relations and coarse-grained temporal signals, a simpler aggregation scheme is often more robust than the full relation-specific parameterization. The contribution of this study therefore lies not only in proposing a learner model that jointly models long-term interests and short-term interests, but also in revealing the data conditions under which this method is applicable, thereby providing insights for applying LM-GNN to platforms that offer richer behavioral signals.

Table 5.4: Comparison with Baseline Methods and LM-GNN Variants

Methods	HR@5		HR@10		HR@20		NDCG@5	NDCG@10	NDCG@20
	Eq. 1	Eq. 2	Eq. 1	Eq. 2	Eq. 1	Eq. 2			
DARL [15]	63.12%		77.63%		87.71%		48.53%	53.25%	55.30%
ACKRec [6]		64.70%		81.22%		92.55%	46.35%	51.70%	54.59%
AGNN [29]							30.22%	33.36%	35.86%
KGNN [27]							26.03%	30.20%	34.34%
TP-GNN [22]							15.51%	20.83%	23.91%
LM-GNN Variant 1	2.24%	4.04%	5.58%	9.91%	12.16%	21.03%	1.73%	3.27%	5.56%
LM-GNN Variant 2	10.96%	18.88%	19.09%	31.62%	30.05%	46.48%	9.54%	13.07%	16.67%
LM-GNN Variant 3	2.34%	4.24%	5.80%	10.28%	12.38%	21.36%	1.81%	3.38%	5.66%
LM-GNN Variant 4	2.28%	4.13%	5.65%	10.04%	12.29%	21.20%	1.76%	3.31%	5.61%
LM-GNN Variant 5	1.77%	3.22%	4.24%	7.65%	9.64%	16.93%	1.44%	2.54%	4.42%
LM-GNN Variant 6	2.35%	4.25%	5.94%	10.51%	12.55%	21.59%	1.83%	3.49%	5.76%
LM-GNN Variant 7	10.72%	18.46%	18.71%	31.05%	29.76%	45.99%	9.33%	12.80%	16.42%

6 Conclusion and Future Work

6.1 Conclusion

This thesis proposed LM-GNN, a learner model for personalized course recommendation in online learning platforms. Different from ConceptGCN[2] that rely mainly on a single user-item interaction type, LM-GNN is built on learner-specific relationships and emphasizes multi-relationship learner modeling. In particular, the proposed method integrates three learner-specific relations, namely (Learner, *DNU*, Concept), (Learner, *INTERESTED_IN*, Concept), and (Learner, *ENGAGED_IN*, Course), in order to capture complementary aspects of learner behavior, including knowledge state, concept-level interest, and course-level engagement.

To support this goal, this thesis designed a learner modeling framework that combines a heterogeneous knowledge graph, graph neural networks, and SBERT-based semantic representations. Based on this design, LM-GNN constructs learner representations from historical interactions under different learner-specific relations and incorporates both long-term interests and short-term preferences into the learner model. More specifically, long-term interests are modeled through a relation-specific sequential matrix that combines semantic correlation among interacted nodes with the temporal order of interactions, while short-term preferences are modeled through relation-specific position weights that emphasize more recent interactions. The resulting learner embedding is then matched with course embeddings from CourseMapper and MOOCCentral to generate personalized course recommendations.

In addition to the method design, this thesis also examined the applicability of LM-GNN under different data conditions through experiments on the MOOCCube dataset. Since MOOCCube does not provide the same richness of learner-specific relations and fine-grained behavioral timestamps as CourseMapper, the original learner model had to be adapted to fit the available relations. The experimental results showed that LM-GNN did not outperform existing supervised baselines on MOOCCube. However, this result should not be interpreted as evidence that the learner model design is invalid. Rather, the findings suggest that the effectiveness of LM-GNN is related to the richness of learner-specific relationships, the reliability of behavioral signals, and the granularity of temporal information. In particular, the discussion and ablation study indicate that when only limited and coarse-grained relations are available, the advantages of multi-relationship learner modeling and short-term preference modeling may be more difficult to observe.

Overall, this thesis presents a learner-specific and multi-relationship learner modeling framework for course recommendation. It also offers some initial observations on the conditions under which such a model may be more effective.

6.2 Future Work

There are several directions for future work.

First, LM-GNN should be further evaluated with real users on the CourseMapper platform through a user study. While this thesis provides offline evaluation and discussion, a learner model based on learner-specific relationships should also be assessed in an authentic learning scenario. A future study could invite learners to use the platform, receive personalized course recommendations, and provide feedback through questionnaires. Such a study would help evaluate the practical usefulness, relevance, and overall user satisfaction of the recommendation results.

Second, future work should integrate the recommendation results more fully into the front-end of the CourseMapper platform. In addition to generating recommendations, the system should present them through a clear and informative interface. For example, the recommended courses could be displayed together with their source platform, institution, instructor, course description, and direct course link. This would improve the usability of the system and support learners in making better-informed choices.

Overall, future work should focus on both real-world evaluation and more complete system deployment on CourseMapper.

List of Tables

3.1	Learner-specific relation timestamp records for online modeling	17
5.1	Statistics of the MOOCCube dataset	52
5.2	Entities in the MOOCCube Dataset	52
5.3	Training data, test data, and Top-5 recommendation lists	60
5.4	Comparison with Baseline Methods and LM-GNN Variants	69

List of Figures

3.1	The Conceptual Architecture of LM-GNN	10
3.2	Personal knowledge graph in the CourseMapper database.	11
3.3	Knowledge graph schema of the MOOCCentral database.	12
3.4	An example of historical interactions under multiple learner-specific relations. (a) shows the PKG before the first automatic recalculation. (b) shows the PKG after the first automatic recalculation. (c) shows the PKG before the second automatic recalculation. (d) shows the PKG after the second automatic recalculation. The label on each edge indicates the relation type and its corresponding timestamp. For the DNU relation, the edge represents whether the concept is marked as not understood. For the interest relation, the edge also shows the learner's interest score for the concept, whereas for the engagement relation, the edge shows the learner's engagement level for the course.	26
4.1	Technical architecture	27
4.2	learner model embedding stored in neo4j.	41
5.1	The initial PKG in MOOCCube.	53
5.2	The final PKG in MOOCCube.	54
5.3	The enrolled course count distribution on training set and testing set	65

Bibliography

- [1] Qurat Ul Ain et al. "Automatic construction of educational knowledge graphs: a word embedding-based approach". In: *Information* 14.10 (2023), p. 526.
- [2] Rawaa Alatrash et al. "ConceptGCN: Knowledge concept recommendation in MOOCs based on knowledge graph convolutional networks and SBERT". In: *Computers and Education: Artificial Intelligence* 6 (2024), p. 100193.
- [3] Krisztian Balog and Tom Kenter. "Personal knowledge graphs: A research agenda". In: *Proceedings of the 2019 ACM SIGIR International Conference on Theory of Information Retrieval*. 2019, pp. 217–220.
- [4] Vic Degraeve et al. "R-GCN: The R could stand for random". In: *arXiv preprint arXiv:2203.02424* (2022).
- [5] Xavier Glorot and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2010, pp. 249–256.
- [6] Jibing Gong et al. "Attentional graph convolutional networks for knowledge concept recommendation in moocs in a heterogeneous view". In: *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 2020, pp. 79–88.
- [7] Xiangnan He et al. "Fast matrix factorization for online recommendation with implicit feedback". In: *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. 2016, pp. 549–558.
- [8] Xiangnan He et al. "Lightgcn: Simplifying and powering graph convolution network for recommendation". In: *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 2020, pp. 639–648.
- [9] Xiangnan He et al. "NAIS: Neural attentive item similarity model for recommendation". In: *IEEE Transactions on Knowledge and Data Engineering* 30.12 (2018), pp. 2354–2366.
- [10] Xiangnan He et al. "Neural collaborative filtering". In: *Proceedings of the 26th international conference on world wide web*. 2017, pp. 173–182.
- [11] Xiangnan He et al. "Trirank: Review-aware explainable recommendation by modeling aspects". In: *Proceedings of the 24th ACM international on conference on information and knowledge management*. 2015, pp. 1661–1670.
- [12] Cheng Hsu and Cheng-Te Li. "Retaggn: Relational temporal attentive graph neural networks for holistic sequential recommendation". In: *Proceedings of the web conference 2021*. 2021, pp. 2968–2979.
- [13] Eleni Ilkou. "Personal knowledge graphs: Use cases in e-learning platforms". In: *Companion Proceedings of the Web Conference 2022*. 2022, pp. 344–348.

- [14] Santosh Kabbur, Xia Ning, and George Karypis. "Fism: factored item similarity models for top-n recommender systems". In: *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2013, pp. 659–667.
- [15] Yuanguo Lin et al. "Adaptive course recommendation in MOOCs". In: *Knowledge-Based Systems* 224 (2021), p. 107085.
- [16] Haokai Ma et al. "Negative sampling in recommendation: A survey and future directions". In: *ACM Transactions on Information Systems* (2024).
- [17] Wenjun Ma, Yibing Zhao, and Xiaomao Fan. "Cascaded knowledge-level fusion network for on-line course recommendation system". In: *IEEE Transactions on Big Data* 10.4 (2023), pp. 457–469.
- [18] Nils Reimers and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks". In: *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.
- [19] Michael Schlichtkrull et al. "Modeling relational data with graph convolutional networks". In: *European semantic web conference*. Springer. 2018, pp. 593–607.
- [20] Markus Schröder, Christian Jilek, and Andreas Dengel. "A human-in-the-loop approach for personal knowledge graph construction from file names". In: *Third International Workshop on Knowledge Graph Construction*. 2022.
- [21] Martin G Skjæveland et al. "An ecosystem for personal knowledge graphs: A survey and research roadmap". In: *AI Open* 5 (2024), pp. 55–69.
- [22] Jingjing Wang et al. "Top-N personalized recommendation with graph neural networks in MOOCs". In: *Computers and Education: Artificial Intelligence* 2 (2021), p. 100010.
- [23] Qilong Yan et al. "Relation-aware heterogeneous graph for user profiling". In: *Proceedings of the 30th ACM international conference on information & knowledge management*. 2021, pp. 3573–3577.
- [24] Hong Yang. "Sequence Recommendation of Knowledge Concepts using Graph Neural Networks". In: (2024).
- [25] Yajing Yang et al. "An explainable graph-based course recommendation model based on multiple interest factors". In: *Expert Systems with Applications* 264 (2025), p. 125889.
- [26] Jifan Yu et al. "MOOCCube: A large-scale data repository for NLP applications in MOOCs". In: *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2020, pp. 3135–3142.
- [27] Huanyu Zhang et al. "KGAN: Knowledge grouping aggregation network for course recommendation in MOOCs". In: *Expert Systems with Applications* 211 (2023), p. 118344.
- [28] Jing Zhang et al. "Hierarchical reinforcement learning for course recommendation in MOOCs". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. 01. 2019, pp. 435–442.
- [29] YUNCHAO ZUO, HAO LUO, and LITAO XU. "Enhancing MOOCs Personalized Recommendation with Graph Neural Networks and Attention Mechanisms". In: (2023).

Appendices

Appendix 1

```
1 for($i=1; $i<123; $i++)  
2 {  
3     echo "work harder! ;)";  
4 }
```